

University of Glasgow | School of Computing Science

A Reproducible Testbed for Media over QUIC Transport: Interop, Controlled Impairments and Baselines

Hardik Hemant Pandey

School of Computing Science
Sir Alwyn Williams Building
University of Glasgow
G12 8QQ

A dissertation presented in part fulfilment of the requirements of the Degree of Master of Science at The University of Glasgow

10th September 2026

Abstract

This dissertation builds a reproducible Containernet and Docker testbed for Media over QUIC Transport. It pins moq-rs, imquic and moxygen plus LL-DASH over HTTP/2 (H2) and HTTP/3 (H3), all playing the same 120 second film with 2 second groups. Loss, bandwidth, delay and queue are set per link with kernel receipts, under static limits only. Each viewer is scored on byte value with strict draft tiers, and the worst viewer sets the verdict. Headline, loss, fanout, fairness and scale rows use three repeats. H3 gains about 37 pct over H2, and MoQ delivers near full cover near 2.3M, about 1.95x H3, but fails byte proof on most long rows. MoQ fanout serves one viewer and starves the rest at every size on the d18 pin while d16 shares fate and H3 serves all, a pin regression not a protocol limit. Join cost and wire time stay null throughout and transport is read from finish time plus via tags. Delay, bandwidth, queue and pilot rows with one run are appendix only. Findings cover the integrity gap at high cover, the clock split, the file only fanout rule, and the single consumer relay gap.

Education Use Consent

I hereby give my permission for this project to be shown to other University of Glasgow students and to be distributed in an electronic format. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Name: Hardik Hemant Pandey Signature: 

Acknowledgements

I thank my supervisor Prof. Colin Perkins for guidance on scope and method, the MoQ community for open code and drafts, and my family and friends for steady support during the build and write up. I managed my references with Zotero. I learned LaTeX from Jake B on YouTube at `https://www.youtube.com/watch?v=lgiCpA4zzGU`, which taught me how to use it. I drew my flowcharts and diagrams with Excalidraw and PowerPoint. I acknowledge the use of AI tools, including Deepseek V4 Flash, Muse Spark 1.2, Gemini and Grammarly, for writing assistance, scaffolding, log triage, figure generation from text pointers I supplied, and spelling checks. All citations were collected and verified by me, and all measured claims are backed by run artefacts in the submitted code; AI output was never used as evidence without verification, as detailed in Chapter 7.

Contents

1 Introduction **6**

2 Survey **7**

2.1 Why Media over QUIC (MoQ) Transport exists 7

2.2 How does MoQT work / MoQT working explained 7

2.3 Draft Evolution 9

2.4 Implementations in Scope 10

2.5 Baseline comparison 10

2.6 Prior Testbeds and Interop Testing 10

3 Problems, Requirements, and Solution Overview **13**

3.1 Problem Statement 13

3.2 Research Questions 13

3.3 Solution Requirements and Overview 14

3.3.1 Solution requirements 14

3.3.2 Solution overview 15

4 Design Methodology **16**

4.1 Design principles and requirements trace 16

4.2 Experimental environment and comparison basis 19

4.3 Correctness, measurement availability and limits 20

4.4 Lessons from the design phase 21

5 Implementation **22**

5.1 Registry as single source of truth 22

5.1.1 How draft agreement is checked 22

5.2 Topology descriptors 23

5.2.1 Where weather goes, with an example 23
5.3 Harness modules 23
5.3.1 How scoring and timing work 24
5.4 Test plans and media 24
5.5 Known build-time limits 25

6 Evaluation **26**
6.1 Verification 26
6.2 Interop matrix 26
6.3 Performance 27
6.4 Chain status 30
6.5 Chapter summary 30

7 How effective is AI for these tasks **31**
7.1 Where AI was used 31
7.2 Performance by task 31
7.3 How AI output was verified 31
7.4 Lessons for future work 32

8 Discussion **33**
8.1 Findings and meaning 33
8.2 What held, what did not, and what was left out 33
8.3 If I started today 34

9 Conclusion **35**
9.1 Future work 35

A Build and evidence index **36**
A.1 Build pins and commands 36
A.2 Evidence index 36
A.3 Commands I ran 36

B Sweep and pilot tables **38**

B.1 Loss, delay and bandwidth sweeps 38

B.2 Queue and bufferbloat 38

B.3 Fanout, fairness and chain single runs 38

B.4 Pilot rows 38

C Raw evidence excerpts **40**

C.1 Headline aggregate rows 40

C.2 Fanout scale rows 40

C.3 Shaping receipt 41

C.4 DASH sidecar proof 41

C.5 Relay log proof 41

Bibliography **42**

Chapter 1: Introduction

A huge chunk of live media delivered over the open internet uses DASH/LL-DASH (broadcast delivery) which is HTTP media streaming, and WebRTC (interactive / ultra-low latency) which is a real-time media stack. DASH scales to millions of users but at the cost of seconds of buffering delays while WebRTC achieves sub-second latency but is designed for small peer-to-peer sessions (like video calls, live collaboration, etc.) rather than large scale distributions. The IETF Media Over QUIC (MoQ) working group is standardising a transport protocol that aims to dissolve this trade-off by combining media delivery and real-time communication over QUIC [1]. Its MoQ Transport protocol is currently at draft-20 [2] as of August 2026, and several independent implementations have emerged. Each claims conformance with the draft, yet such claims rest largely on self-reported feature lists rather than verified behaviour.

Existing evidence of compatibility comes from periodic interop testing events, which demonstrate that sessions can succeed under ideal network conditions. What these events do not show is whether implementations negotiate the features they claim to support or how they behave when bandwidth, delay, and packet loss are deliberately varied. Oftentimes in existing testbeds, network conditions are often uncontrolled, topologies are fixed, and outcomes are just a success/failure binary with little to no link between the verdict and protocol level evidence.

This dissertation addresses that gap with the help of a reproducible testbed [3] which emulates configurable network topologies using Containernet and Docker, executes publisher-relay-subscriber combinations across implementations, and applies scripted impairments to individual links. Media delivery is measured per subscriber directly from protocol logs, yielding auditable pass/partial/fail verdicts.

Chapter 2 introduces the need for MoQT, explains key concepts, and surveys previous work including testbeds and their gaps which would help the audience to gain a better understanding on the topic. Chapter 3 talks about the problem and requirements, and gives a brief overview of the solution in this dissertation. The detailed design and method are in Chapter 4. The build and results are in Chapters 5 and 6 respectively while Chapter 7 talks about how AI performs in this work followed by Chapter 8 with key choices plus what could be better plus my notes on how I would start anew. The close and future work are in Chapter 9.

Chapter 2: Survey

This chapter surveys the background necessary to understand the research questions addressed in my dissertation in Chapter 3. It first explains why a new media transport is needed and then proceeds to introduce MoQT at a sufficient level to understand about the impairments and topologies, and their effect on the measured quantities. It traces major draft evolutions, introduces the three systems under test, and finally reviews prior testbeds to pin down gaps that the present work addresses.

2.1 Why Media over QUIC (MoQ) Transport exists

Large-scale live streaming commonly uses HTTP-based delivery such as DASH [4] and HLS [5]. Video is cut into small files that can be copied by standard CDN caches to millions of viewers. This scales very well, but waiting to fill, deliver and buffer each file adds seconds of delay, typically 5–30 s end to end. Low-latency variants reduce this to a few seconds, yet they still wait for segments and stall when the network does.

Video calls take the opposite approach. Systems such as WebRTC send media directly with very little buffering, so delay is often well under a second. This works well for small interactive groups, but reaching a broadcast-size audience this way needs large numbers of helper servers to copy each stream to every viewer, which is costly and fragile.

Media over QUIC Transport (MoQT) is motivated by this trade-off. It keeps the CDN-style idea of middleboxes that copy and share data, but runs it over QUIC [6], which supports stream multiplexing with per-stream flow control and loss recovery. Late data can be skipped rather than holding up fresh data, so playback can stay close to the live edge while still scaling. It also aims to use one protocol from ingest to delivery, rather than separate protocols for contribution and distribution. The next section introduces how MoQT is structured, at the level needed to understand the impairments and topologies evaluated in Chapter 6.

The trade-off is therefore scalability versus latency:

- WebRTC gives low-latency direct paths, but broadcast scale needs many SFUs. Comparative load testing of open-source SFUs in single-server configurations [7] shows this scaling path means provisioning and managing such servers.
- LL-DASH/HLS scale via HTTP and CDNs by caching one encoded stream at the edge. LL-DASH uses CMAF chunked packaging and HTTP chunked transfer encoding to reduce latency [8]. Nevertheless, achieving lower latencies in a scalable and reliable way motivated a QUIC-based solution [8].

2.2 How does MoQT work / MoQT working explained

Here are the Key components of MoQT:
1. Object - It is the smallest unit of media/application data transported by MOQT. It contains a piece of data, such as a video frame or audio sample. It has metadata such as an Object ID. Objects are organized into subgroups and groups. For Example: one encoded video frame

can be an object.
2. Group - It is a collection of subgroups and/or objects that belong together in a logical sequence. Groups are useful for identifying a particular segment or generation of media.
3. Subgroup - It is a smaller organization within a group. It contains objects that can be sent together and share ordering/dependency characteristics. Subgroups can help with prioritization.
4. Track - It represents a continuous stream of related objects. Examples include video track, audio track, screen-sharing track, subtitle track, etc. A track has a Track Name and is associated with a namespace. For simpler understanding, a track is a collection of groups.
5. Publisher - It is the participant that makes tracks available to other participants. The publisher produces and sends the media objects.
6. Subscriber - It is the participant that requests and receives tracks from a publisher. A subscriber can subscribe to a particular track and receive its objects.
7. Relay - It sits between publishers and subscribers and forwards MOQT data. It can reduce the need for a publisher to send the same data independently to many subscribers.
Refer to Figure 2.1 for a visualized explanation of tracks and its contents. The broad architecture of how MoQT works is represented in Figure 2.2

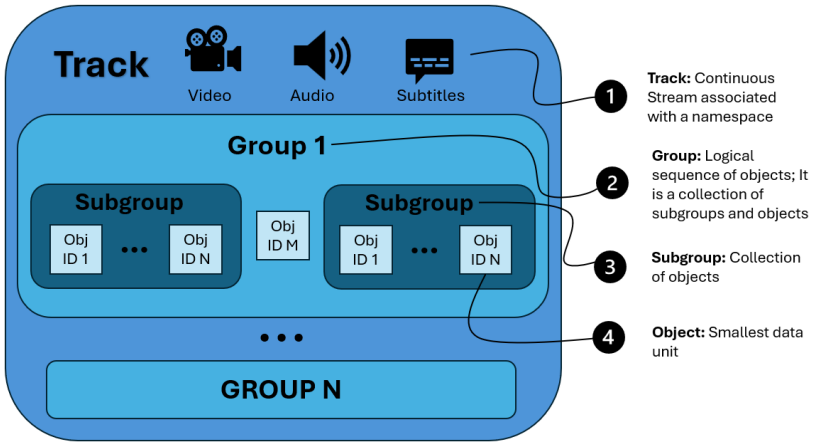


Figure 2.1: Tracks in MoQT

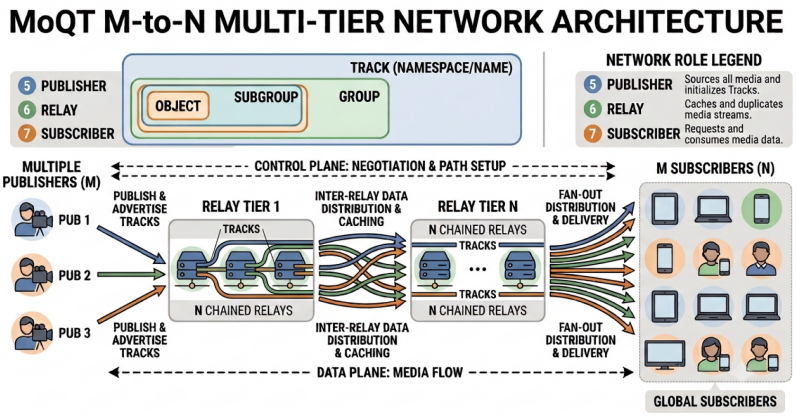


Figure 2.2: MoQT Broad Architecture. Schematic made with Gemini from my notes, no measured data.

2.3 Draft Evolution

MoQ Transport (draft protocol) has come a long way from when it first started in Feb 2022. Infact, the MoQT is evolving so rapidly that the draft version when I started working on this dissertation, was 18 and the latest draft version (as of 10th September 2026) is draft 20 [2]. Table 2.1 summarises the key changes from the draft changelog [2] and comparison with WebRTC / LL-DASH.

Table 2.1: Media over QUIC Transport draft evolution (00–20), from the draft changelog [2]

Version	**Date**	**Major changes**	**Vs. WebRTC / LL-DASH**
lcurley-00	May 2023	Merge of WARP, RUSH, QuicR	Single QUIC media starting point
ietf-00	Jul 2023	WG adoption; SETUP, Track/Group/Object, QUIC + WebTransport, relays	One protocol ingest + delivery; CDN fan-out vs. SFU
01–07	2023–2024	Subgroup model, stream-per-subgroup, DATAGRAM vs. stream, SUBSCRIBE/FETCH	No TCP HoL blocking; cache fill
08–16	2025–Jan 2026	REQUEST_UPDATE, GOAWAY, priorities (§7), DELIVERY_TIMEOUT, ALPN-only	Drop stale under congestion; simpler than ICE/DTLS
17	Mar 2026	0-RTT subscribe, SUBSCRIBE_TRACKS split, 7-byte varint	Lower startup RTT
18	May 2026	moqt:// URI, REDIRECT, FIRST_OBJECT, OBJECT + SUBGROUP timeouts, padding	Implementable staleness; interop target
19	Jul 2026	Range Filters, concurrent subs/Track, SKIPPED, FIN vs. RST	SVC layering; partial reliability
20	Aug 2026	FILL_PARAMETERS replaces Joining FETCH, LOCATION_FILTER, Type Flags	1-RTT join + live; hybrid stream/datagram

Table 2.2 explains the progress made on goals which MoQT eventually plans to achieve.

Table 2.2: MoQT draft-20 progress toward unifying low latency and scale

Goal	**WebRTC / LL-DASH today**	**MoQT-20**	**How far + gap**
Sub-second latency	WebRTC 0.05–0.3s; LL-DASH 2–6s	QUIC streams, datagram+stream mix, timeouts	Close on wire; needs ABR + tuning
CDN scale	DASH millions via HTTP cache; WebRTC 100s/SFU	Relay cache, aggregation, Alias	Ready on paper; no million-viewer proof
Fast join	WebRTC multi-RTT ICE/DTLS; DASH manifest chain	0-RTT, FILL_PARAMETERS 1-RTT join	Major gain in v20; needs player + UDP-open net
Resilience	WebRTC ICE restart; DASH rebuffer	QUIC migration, GOAWAY, FETCH fill	Better on paper; impls immature
One stack	RTMP ingest to HLS repack	Single moqt:// pub/sub live + VoD	Converged; codecs/encrypt separate

As shown in Table 2.2, MoQT combines WebRTC-like latency mechanisms (QUIC streams/datagrams, OBJECT/SUBGROUP timeouts, 0-RTT/1-RTT `FILL_PARAMETERS` join) with DASH-like scale mechanisms (caching/aggregating relays, single `moqt://` pub/sub). However, ABR (SSTS), monitoring, and companion formats remain split out for WGLC, and million-viewer, WebTransport and player evidence stays at pilot stage, so it is not yet production-ready.

2.4 Implementations in Scope

There are many different implementations of MoQT but they differ in language, maturity, roles offered and media path, so this dissertation fixes three systems under test that jointly span the design space while all advertising the same draft line in their repos (protocol defined in [2]): `moq-rs` (Cloudflare, Rust; relay, publisher and subscriber), `imquic` (Meetecho, C on picoquic; relay and subscriber for media, its publisher being a clock generator excluded from media-delivery tests) and `moxygen` (Meta, C++ on mvfst/proxygen; relay with FLV-based publisher/subscriber clients built from source because the official images ship only the relay). Each is pinned by commit rather than floating tag for reproducibility (`my_testbed` registry; full build provenance in Chapter 5): `moq-rs` draft-18 at `52b8a90` (drafts 14/16 pinned separately at `37e636e7`/`2d16c9c8`), `moxygen` at `4095b18` with a proxygen override at `3ad19ac` for a use-after-free and qmux-compat fix, and `imquic` at `1f4cbf8` on picoquic `0dbd19d`, with the draft (16–19) selected at runtime via `-M` so its four image tags are intentionally identical builds. Other implementations (`moqtail`, `moq-dev`, `quiche-moq`) are documented in the registry but excluded as early-stage or incomplete at this draft. Claims here are as-advertised only; whether negotiated behaviour matches them is measured in Chapter 6.

Selection was limited by build and evaluation time, so this is a representative, experimentally tractable subset varying language, stack and roles, not an exhaustive survey of all implementations at the time of study.

2.5 Baseline comparison

LL-DASH is the natural baseline because it is the system MoQT is most often compared against: video packed as CMAF [9] segments over HTTP and copied by normal web caches, with chunked delivery cutting the long delay of classic segment-buffered DASH (seconds to tens of seconds) down to a few seconds. The comparison is fair where both play the same source: both offer the same video at several quality levels, both must trade picture quality against delay with the switching logic left to the player rather than fixed by either standard, and LL-DASH scales through web caches while MoQT aims for similar scale through relays and caches, so the two most relevant head-to-head studies stage exactly this pairing under the same network settings [8, 10]. Kuo provides a complementary single-stack MoQ latency and bandwidth-estimation grid rather than a same-source pairing [11]. The reason for running the baseline twice: H2 runs over TCP where one loss holds up the rest, while H3 runs over QUIC; MoQ versus H2 therefore compares today's HTTP+TCP path against QUIC objects, and MoQ versus H3 (both QUIC) isolates push-subscribe of skippable objects versus paced segment fetch. Other baselines control less well: WebRTC targets calls rather than cacheable files, and speed tests omit video packaging, so the $\sim$2 s chunk (one segment $\approx$ one group) is my testbed choice, not a protocol property.

2.6 Prior Testbeds and Interop Testing

Prior MoQ testing falls into two categories: structured interop events and academic testbeds. The IETF MoQ working group defines MoQT as a publish/subscribe protocol over QUIC and WebTransport [1, 2]. In the community, independent implementations test pairwise compatibility at a pinned draft in periodic interop events. These events establish that sessions can be set up and media can flow between implementations under controlled, typically clean network conditions, which is their intended purpose: they answer whether implementations agree on the wire protocol, not how streaming quality degrades when the network does. They do not systematically vary link impairments, do not exercise multi-relay topologies

Table 2.3: Prior MoQ testbeds. All use a publisher–relay–subscriber pipeline with `tc` as ground truth; none combines a shared bottleneck with multi-relay fanout and per-subscriber protocol-level evidence.

Testbed	Setup	Impairments	Metrics	Limitation
Gurel et al. MMSys'23 [12]	Warp, `quic-go` fork, 2-machine LAN	`tc`: CASCADE, LTE-train, Twitch	SWMAth/IFA vs. ETP vs. `tc` rate	No DASH baseline; 1 relay
Gurel et al. MMSys'24 [8]	`moq-pub`/relay + `origin.js` + `dash.js`, 3 machines	`tc htb` + `iptables` split	Latency, BWE, skips, lat.–quality	Naive re-subscribe; LCFS only
Nguyen et al. MMSys'25 [10]	`moq-rs` d04 + Axum origin + `tc`-proxy; concurrent players	Loss/delay/rate grid	Latency, buffer, BWE, bitrate	Pinned to d04; 1 relay
Kuo KTH'25 [11]	`moq-rs/js` d08, OptiPlex+NUC, Ethernet	RTT $50/150/300$ ms $\times$ loss 0.1–2 %; stepped BW	Adj. latency + ANOVA; probe vs. I-frame, Mann–Whitney	No real switch; no jitter/burst; clock caveat
Gurel et al. ICMEW'25 [13]	`moq-relay/pub` (`testsrc`) + React `moq-js`	UNSUB delay 0–3 s; light vs. heavy	ET/TET/AETR; stall + delta distributions	Aligned-GoP assumption; 1 subscriber

with shared bottlenecks and fanout, and report outcomes per test case rather than as per-subscriber quality distributions with protocol-level evidence.

What remains missing, as Table 2.3 shows, is a testbed that makes single-relay behaviour reproducible and checkable at the byte level: prior studies each cover part of the space—initial Warp instrumentation [12], the first MoQ versus LL-DASH comparison [8] and its follow-up [10], a latency and bandwidth-estimation grid [11], atomic track switching [13]—following the origin and relay design exploration in [14], but without published shaping scripts, pinned implementation commits, same-source MoQ and H2/H3 baselines, and per-subscriber byte-value verification with honest partial and error verdicts. Tail distributions under shared bottlenecks, bandwidth estimation, and switching dynamics remain future work. The remainder of this section presents such a testbed [3]: a pinned draft and codebase registry, a documented single-relay topology with exact shaping configuration, same-source MoQ versus H2/H3 comparison, and a per-subscriber integrity and throughput schema in which incomplete delivery reads as `partial` and unwired comparisons fail fast as `error` rather than passing silently.

Short forms used in this thesis

Non-networking readers meet many short forms here. This box defines each once. Later chapters reuse them without pause.

Three-way means MoQ against H2 against H3, three protocols in one test. It never means three repeats. Repeats are always stated as n equals 3, which means the same run three times in a row. A run is one start to finish test. A batch is a set of runs launched together. A row is one line in a results table, often merged from repeats.

Status says if the run finished its checks. Verdict says if the bytes were right. Exact means the hash matches. Good means all bytes arrived with at least 90 pct matching prefix. Mismatch

means full but wrong bytes. Partial means short but correct start. Cover means bytes arrived divided by bytes expected. Quiet stop means MoQ pushed its file and went quiet early. Time limit means DASH fetched paced until the clock ran out. Via tag says which path each part used, H3 or TCP fallback.

BOOT means the box started with no draft proof and never counts as agreed. CONFIRMED means the ALPN word was seen on the wire or the setup test passed. ALPN is the short word each side offers in the handshake to name its draft. The mlog is the relay JSON log of objects made. The qlog is the QUIC timing log. The pcap holds headers only. The receipt is the kernel shaping log per host. Fanout is one publisher to many viewers. First object time and wire time stay null where clocks and decrypt do not allow them. Fair index is Jain's fairness index $(\sum x_i)^2/n\sum x_i^2$ over per-viewer bytes [15]: 1.0 when viewers split evenly, down to $1/n$ when one viewer takes all. Trio means one publisher plus one relay plus one subscriber in interop tests.

Chapter 3: Problems, Requirements, and Solution Overview

Chapter 2 established that MoQ Transport is an evolving draft protocol with multiple independent implementations, and that prior testing leaves two precise gaps: (1) no custom, controlled networks and topologies with per- link impairment control, and (2) no claimed-versus-negotiated verification of protocol capabilities. This chapter formalises the problem arising from those gaps, derives the research questions and solution requirements, and presents a high-level overview of the testbed solution. The detailed design and methodology follow in Chapter 4.

3.1 Problem Statement

Chapter 2 leaves two gaps: experimenters cannot set up their own topologies with per-link control of impairments, and no method checks negotiated behaviour against claimed draft support. This matters now because builds under test are pinned to draft-18 while draft-20 is already published, so version drift is normal. MoQT answers DASH and WebRTC with three ideas: QUIC streams remove TCP queueing delays, small objects with deadlines let stale data be skipped instead of queued, and relays with caching share one stream across many viewers. But each idea raises questions (which quality policy, at what scale, at what tail cost) that are larger than one dissertation. The problem addressed here is narrower: without controlled single-relay networks with checked per-link settings, and verification of claimed support backed by per-subscriber byte checks, the draft-support, interop and delivery-under-limits claims in RQ1 to RQ5 cannot be checked. Quality policy, tail results at shared bottlenecks and WebRTC comparison stay out of scope and go to Chapter 8.

3.2 Research Questions

The following six research questions (RQs) are derived directly from the problem statement and the gaps identified in Chapter 2. Each RQ is answered in Chapter 6, with the primary chapter indicated in brackets.
RQ1: Which draft-18/19 features does each implementation support, and what is the highest draft version it negotiates? [Ch6.1]
RQ2: Which (publisher, relay, subscriber) tuples interoperate, and where do they fail? [Ch6.2]
RQ3: How do delivery, throughput and integrity compare for MoQ versus LL-DASH over H2 and over H3 on the same source, with first-object times shown as nulls where clocks and decrypt do not allow them? [Ch6.3]
RQ4: What is the throughput, coverage and integrity under controlled loss and bandwidth limits per link, under static limits only with no rate policy? [Ch6.3]
RQ5: What is the relay impact on delivery, throughput, and fan-out (1 publisher to N subscribers), with join cost attempted but null throughout and transport read from finish time plus via tags? [Ch6.3]
RQ6: What conformance gaps should be reported to the MoQ Working Group? [Ch8]

3.3 Solution Requirements and Overview

3.3.1 Solution requirements

Table 3.1: Solution requirements traced from research questions to realisation and evidence

Req	From	Requirement	Realised in	Evidenced in
R1	RQ1	Pin each build by commit; check claimed support against observed negotiation	Ch4 verifier; Ch5 registry	Ch6.1 tiers
R2	RQ2	Run every tuple under one verdict rule	Ch4 runner; Ch5 harness	Ch6.2 tuple table
R3	RQ3	Same source for MoQ and H2/H3; record delivery plus first-object where measurable, nulls shown	Ch4 metrics; Ch5 three-way	Ch6.3 numbers (nulls shown)
R4	RQ4	Set loss and bandwidth per link under static limits only; record throughput, coverage, integrity	Ch4 shaping; Ch5 sweeps	Ch6.3 sweep rows
R5	RQ5	Run one publisher to N subscribers; record each subscriber	Ch4 fanout plus checks; Ch5 fanout	Ch6.3 per-sub table
R6	RQ1 to RQ5	Reproduce from pins plus config plus logs; pass, partial or fail	Ch4 artefacts; Appendix A	Ch6 summary plus receipts

Each requirement exists because the corresponding question cannot be answered without it, and each evidence column is trusted for one stated reason: R1 pins every implementation by commit and checks claimed support against observed negotiation because version drift is normal, and the tiered verdict is trusted because it quotes the ALPN and messages seen in that run; R2 runs every tuple on one topology under one verdict rule because mixed setups make interop tables unreadable, and the tuple table is trusted because the same rule scores all cells with failures described, not dropped; R3 uses the same source for MoQ and H2/H3 and records first-object time with delivery because different inputs would explain any gap, and the numbers are trusted because nulls are reported as nulls rather than filled in; R4 applies loss and bandwidth limits per link and records throughput, loss and delivery because behaviour under limits is the claim being tested, and the sweep rows are trusted because each carries its exact shaping audit log while claiming static limits only and no rate policy; R5 runs one publisher to N subscribers with per-subscriber byte checks because fanout is the scaling claim, and the per-subscriber table is trusted because the worst subscriber sets the verdict while the independent last-mile shaping is labelled as such; R6 demands pins plus config plus logs with pass, partial or fail because otherwise no run can be rechecked, and it holds because unwired comparisons fail fast instead of passing silently. In MoSCoW terms all six are Must, since each RQ is unanswerable without its row, while Should and Could items sit in Chapter 8 as future work and WebRTC comparison is an explicit Won't for this study.

3.3.2 Solution overview

The testbed meets these requirements in this way: the registry fixes the code, the plan fixes the test, the topology fixes the network, the runner fixes the run, verification fixes the verdict, and the report fixes the record, with detail in Chapter 4 and build provenance in Chapter 5. Figure 3.1 previews the four wirings defined in the testbed. Basic and fanout are citable; chain and chainfanout are exploratory only.

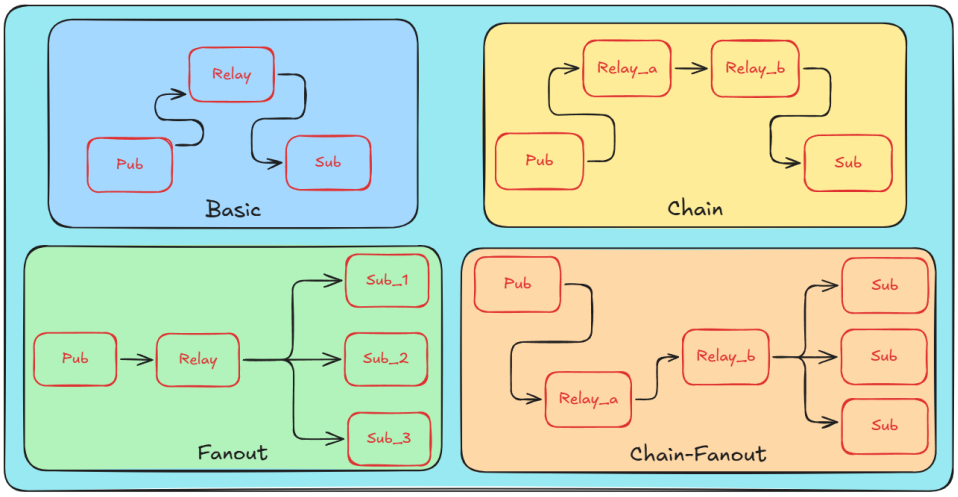


Figure 3.1: Topologies under test: basic, chain*, fanout, and chainfanout* (*exploratory: chain 2/6, chainfanout single campaign 0/5). Shaping and verdicts are defined in Chapter 4.

These wirings are not just random: each isolates one part of MoQT's scale-versus-latency claim. Basic (publisher–relay–subscriber) is the correctness baseline for interop (R2) and for delivery under limits (R3, R4) on a single relay leg. Chain (publisher–relay_a–relay_b–subscriber) adds relay-to-relay forwarding as an exploratory topology to test cache fill and inter-relay impairment without changing the verdict rule. Fanout (one publisher to N subscribers) tests the scaling claim directly: each subscriber is scored separately on byte value and the worst sets the run verdict, so one healthy viewer cannot hide starving ones. Chainfanout (fanout downstream of relay_b) was attempted once as an exploratory campaign only (0/5, best delfrac 0.03); it is not a citable topology and is reported as an honesty box in Section 6.4, with the coordinator-gated hypothesis and future work there.

Chapter 4: Design Methodology

Recall from Chapter 2: live video today uses two different tricks. DASH packs video into lunch boxes (segments) that ordinary web caches can copy to millions. That scales well but arrives seconds late. WebRTC sends each fresh slice (frame) directly with little buffering. That is fast but it does not scale to broadcast size audiences. Media-over-QUIC Transport (MoQT) tries to keep both properties: small addressed pieces (objects in groups, carried as named tracks) forwarded by middleboxes (relays) over QUIC, where stale pieces can be skipped instead of blocking fresh ones. A publisher makes a track, a relay copies it, and each subscriber holds one subscription.

Chapter 3 showed why that promise is hard to check. Implementations advertise the same draft line (protocol defined in [2]) but version drift is normal, and prior interop events show only that a session *can* succeed on a clean network. They do not show what was actually negotiated, nor what happens when the network turns bad. This chapter states *how* the testbed of [3] closes that gap and *why* each choice is the checkable one. How the testbed is built is deferred to Chapter 5. What it measured is deferred to Chapter 6.

Three terms need short glosses for what follows. A *topology* is who talks to whom (for example publisher to relay to subscriber). An *impairment* is bad weather put on one wire on purpose: less bandwidth (a thinner road), added delay (a longer road), or loss (some letters thrown away). A *pin* is the exact recipe version of the software (a commit hash, not a floating tag), so a run can be repeated. Figure 4.1 shows the flow. Table 4.1 traces each requirement R1 to R6 from Table 3.1 to its design section.

4.1 Design principles and requirements trace

Every choice below obeys three principles, in order. Where they conflict, the earlier wins. Each principle is given first in plain terms, then as an enforceable rule.

1. **Repeat from recipe plus plan plus receipts.** *Plainly:* anyone should get the same result from the same ingredients and weather report. *Rule:* a run is defined by pinned commits in `registry.json`, a declarative plan in `testplans/*.yaml` (which programs run, on which topology, with what weather), and stored receipts (`logs/`, `qlogs/*.mlog`, `tc_*.log`). Floating tags are rejected. One subtlety matters. The `imquic` relay is left offering *all* drafts it knows (its default `any`), because forcing it to offer only draft 18 makes it refuse `moq-rs` peers outright (close 271). That refusal is a strictness in the demo tool, not proof that the protocols disagree. Each run still *declares* one draft through its subscriber flag. What was actually agreed (the ALPN token, which is the common language picked during the handshake) is quoted from relay logs in Chapter 6.

2. **One scoreboard, blanks left blank.** *Plainly:* every team plays by the same referee, and where the stopwatch was missing I write "unknown" rather than inventing a time. *Rule:* every subscriber is scored on the *value* of bytes received (hash plus prefix match, not only the byte count), under one ladder: broken checks give `fail`; too few bytes (below 95%) give `partial`; full but wrong bytes give `fail` when integrity is required. The worst subscriber sets the run verdict, so one happy viewer cannot hide

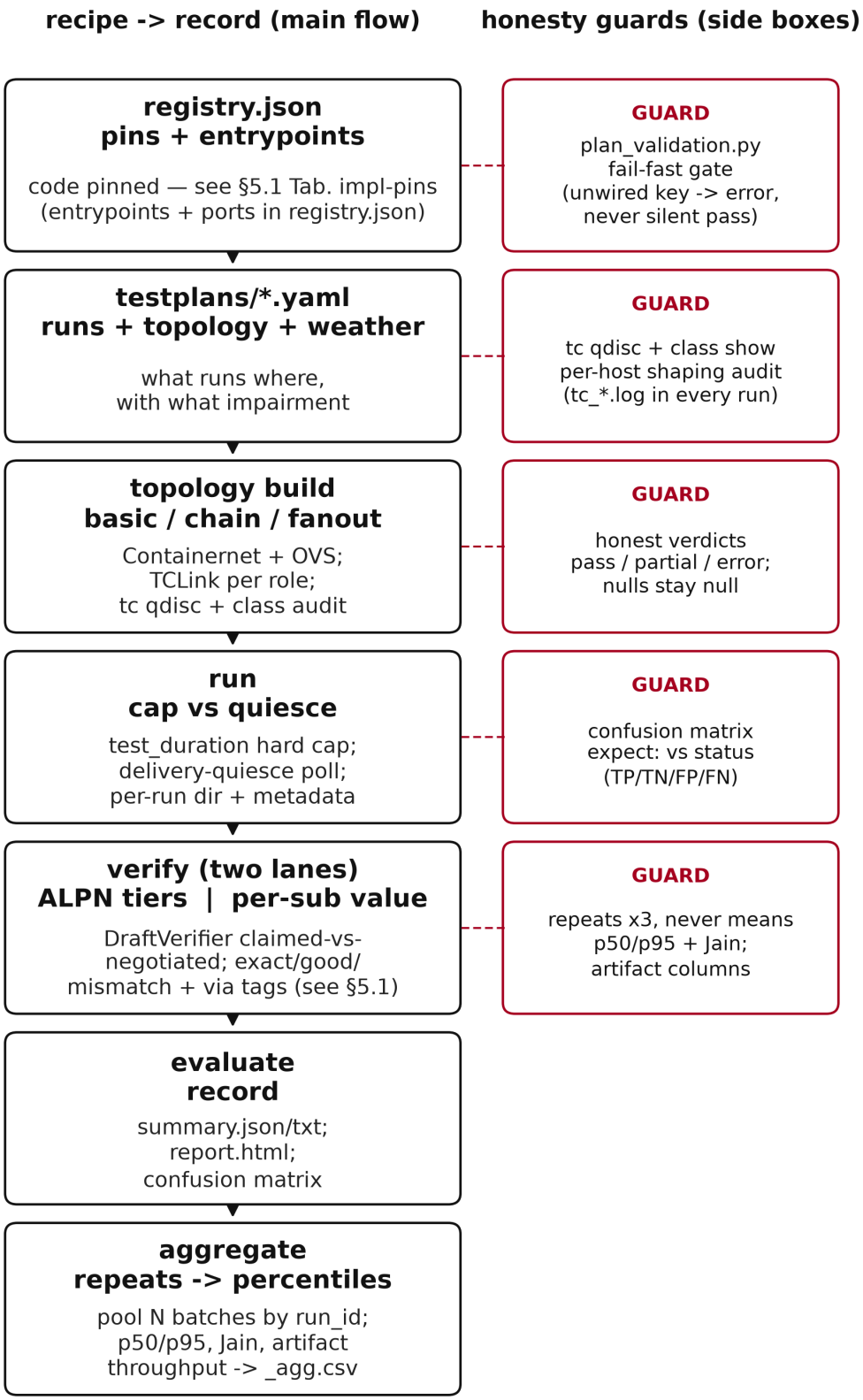


Figure 4.1: Harness flow from recipe to record. The registry pins the code, the plan declares the test, the topology builds the network, verification scores every subscriber on byte value, and repeats aggregate to percentiles. Side boxes are the honesty guards.

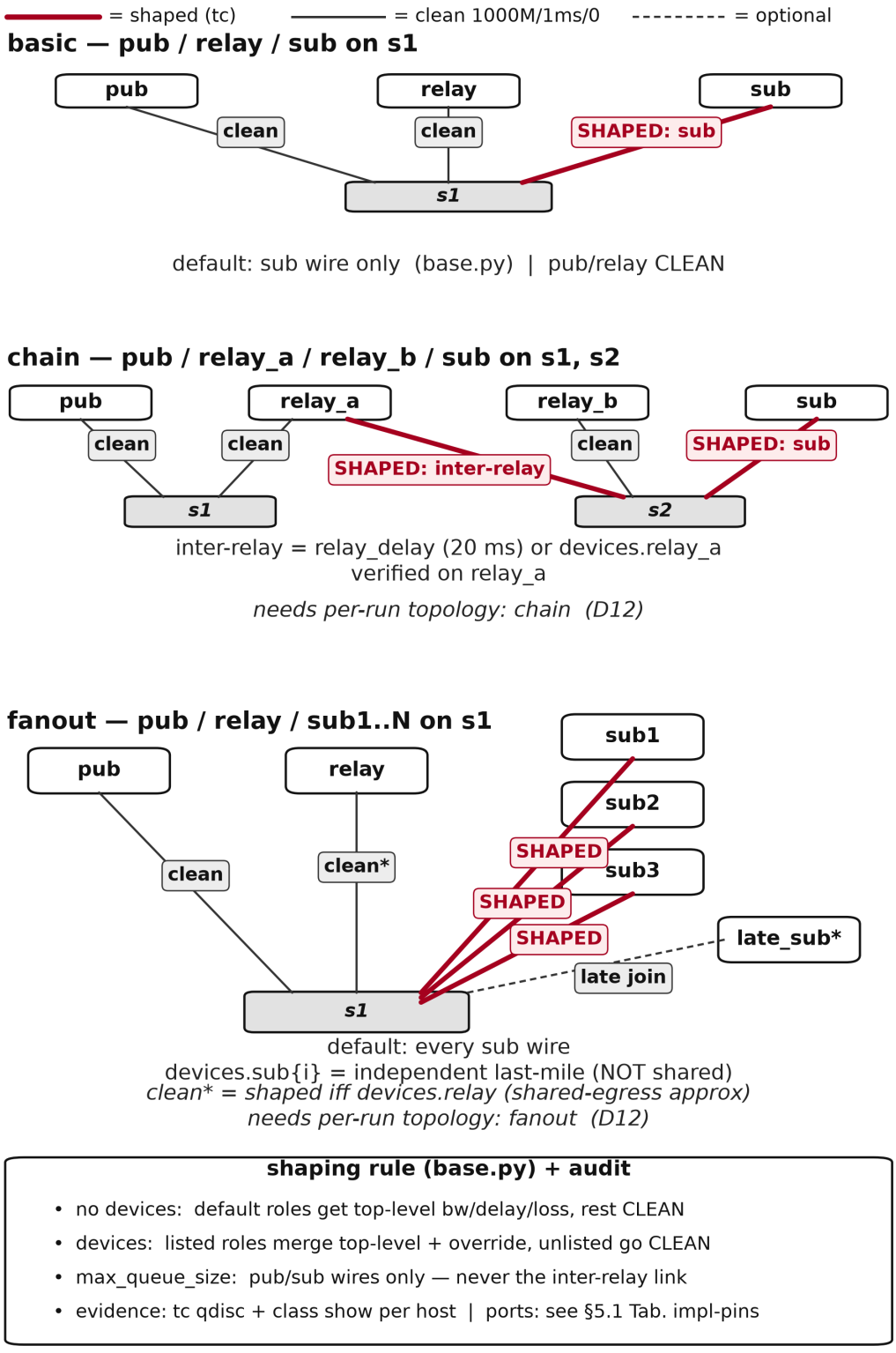


Figure 4.2: Who talks to whom, and which wire carries the weather. Thick wires are shaped by `tc`; thin wires are clean 1000M/1ms links. `devices.*` moves the impairment onto the named wire; anything unwired fails fast instead of running clean silently.

two starving ones. Timings that need synchronised clocks across machines (first object time) or decrypted wire captures are reported as `null` where those conditions fail. For multiviewer MoQ rows only artefact bytes plus verdict are citable, because the relay internal counters double count when subscribers restart (a factor of 2 to 4 inflation observed) while saved files do not.

3. **Every weather claim keeps its receipt.** *Plainly:* if I say "at 1% loss" then I must show which wire was made lossy and that the shaping was really on. *Rule:* by default only viewer wires are shaped. The publisher and relay stay on a fast clean link so extra delay is not silently doubled. Named overrides (`network.devices`) reshape only the named wire and are merged with the top level weather. Everything else stays clean (`topologies/base.py`). Queue size applies to publisher and viewer wires only. Each run stores the kernel shaping receipt `tc_{host}.log` [16]. Two labels prevent the common misreading. Shaping each viewer separately is *independent last miles*, not a shared bottleneck. The shared bottleneck approximation is shaping the relay own wire, which every viewer traverses. Chain rows that claim a relay to relay bottleneck must shape `devices.relay_a`, or that middle wire stayed clean.

Table 4.1 is the contract: what this chapter designs, what Chapter 5 builds, and what Chapter 6 must prove. If a Chapter 6 table cannot point to a row here, then its claim is out of scope.

Table 4.1: Requirements trace in plain terms: what Ch4 designs, Ch6 proves

Req	What this means (design in this chapter)	Proved in
R1	Fix each program version. Quote the language actually agreed, not claimed	Ch6.1 tiers
R2	Play every publisher to relay to subscriber trio on one field, one referee	Ch6.2 tuple table
R3	Feed MoQ and both DASH baselines the same 120 s film. Count from arrival; first-object nulls shown	Ch6.3 three-way plus `completion_reason`
R4	Put bad weather on named wires only. Keep the receipt	Ch6.3 sweeps with `tc` logs
R5	One publisher to N viewers. Score each viewer, worst sets verdict	Ch6.3 per-viewer table
R6	Repeat from recipe plus plan plus logs. Unwired plans fail loud as `error`, never silent pass	Ch6 methods note

4.2 Experimental environment and comparison basis

Network construction :
Each part of the test (sender, relay, viewer) runs in its own sealed box on a pretend network drawn in a file, where each cable is given a fixed speed, delay, loss and waiting-room size, because only a written-down network can be run twice with the same result and lets me point at the cable that caused it. This was a good choice for my single-relay tests: the three wirings come from one description each, every run saves the shaping receipt, and the same test file gives the same verdict on a rerun. Its limit travels with every result: shaping applies where data leaves a box with the burst size left unset, so queue numbers describe each box exit rather than one shared waiting room, and the multi-viewer rows are labelled as separate home connections for exactly this reason. Containernet was chosen over Mininet-only, physical machines, or ns-3 because only Docker images plus a file-drawn topology

rerun the same sender-relay-viewer boxes with per-link receipts; in hindsight Mininet-only would have been easier to debug and is what I would start with on a redo (see Section 8.3), so not every choice here claims to be optimal.

Static scope :
Only fixed loss and bandwidth limits are applied per link, with no quality switching policy in the harness, because fixed weather is checkable while a switching policy is a separate research object. This scoping was good: each sweep row carries its shaping receipt and claims static limits only. Its bound is named, not hidden: no jitter, no burst control, no trace replay, which go to Chapter 8 as stated future work. Static limits were chosen over ABR/rate policy, jitter, burst, or trace replay because fixed weather stays checkable per link with a receipt, while each dynamic mechanism needs single-key threading plus a unit test plus proof that time did not allow (see Section 8.2).

Same-source comparison :
MoQ and both baselines play the same 120 second film with the same two second quantum, because different inputs would explain any gap before the protocols are compared. This was a good control: it separates the delivery path question from the content question, and lets the H2 and H3 pair isolate transport effects from application effects. Its bound is stated with the numbers: MoQ bulk-pushes its file until quiescence (no further progress) while the baselines fetch paced segments sequentially to the time limit, so throughput compares only within one denominator, and long MoQ files hit a relay ceiling that is reported as a finding rather than hidden. Every throughput cell carries media plus `completion_reason` (quiesced vs. time-limit) plus denominator plus `via` label. LL-DASH was chosen over WebRTC because time allowed one baseline family and LL-DASH is the fair pair—CMAF-over-HTTP copied by web caches against MoQ relay fan-out both test cache-style scale, while WebRTC call/SFU servers test a different model. Both H2 and H3 were run because H2 versus H3 splits the transport gain (TCP head-of-line blocking versus QUIC) so MoQ versus H3 then isolates the application difference on the same transport family.

Reproducibility :
Each program is fixed by commit hash in `registry.json`; pins and start commands are listed in Section 5.1 and Appendix A and are not repeated here. Pins were chosen over floating tags because a tag can move overnight and silently change what was tested. The one exception is the imquic relay offering all drafts while each run still declares one draft through its subscriber flag. What was agreed in each run is quoted from the relay logs in Chapter 6, so the claim here stays as advertised only.

4.3 Correctness, measurement availability and limits

Verdicts :
Every subscriber is scored on the value of bytes received, by hash plus prefix match rather than byte count, with the worst subscriber setting the run verdict, because one happy viewer must not hide two starving ones. This is the strongest strand and no structural change is proposed: a short but correct late capture reads partial rather than fail, a wrong byte reads mismatch, and unwired plans fail fast as error instead of passing silently. Value was chosen over byte-count alone because counts pass wrong bytes, worst-subscriber over average or best because fanout must not hide starving viewers, and fail-fast over silent pass because of the stdout-lie exhibit in Chapter 7.

Null handling :
Where the clocks or the captures could not support a timing, I left it null rather than filling it

in. A null in my tables means the condition for that value did not hold, never zero. Table 4.2 maps each metric, and Chapter 8 names what would unlock the missing ones. For transport I use qlog-only round trip plus finish time with `via` tags instead of interpolation.

Metric limits :
Each metric in Table 4.2 is kept for one reason and bounded in one stated way. A null never means zero; it means the conditions for that value did not hold.

Table 4.2: Why each metric is kept and what a null means

Metric	Why kept	What null or bound means
Delivery and integrity	Counts hide wrong bytes, so bytes plus hash decide	Mismatch means wrong bytes, not short count; partial means short but correct prefix
Throughput	Checks work done per run duration	Compares only within one denominator; MoQ bulk-push to quiesce versus DASH paced-fetch to time-limit differ by design; each cell shows media plus `completion_reason` plus `via` label
First-object time	Join cost is the latency claim under test	Null means clocks across hosts do not allow it; reported, not filled in
Round-trip estimate	Validates shaping against observed transport	qlog value kept; wire value null where decrypt is not available
Per-subscriber split	Fanout claims need each viewer scored	Worst subscriber sets the verdict; shaping per last mile is labelled; fanout cites artefact bytes plus verdict, never relay `throughput_p50`; switch only on `>2x` WARN

4.4 Lessons from the design phase

My biggest lesson was build budgeting plus size. I had to give more space for building images. My VM was allocated 75GB and the dependencies and stuff took around 30 to 50GB so the image failed plus time consumed. Once I waited 82 mins for the build to fail and the huge lump of logs was too big to pinpoint the issue. My fix was making more stages and steps so that older ones can be cached. The Dockerfile which AI created was good but not efficient, so I had to intervene.

The same pattern held elsewhere. Impairments were applied to every device which caused a bug. I gave it the idea to apply per device impairments. Verdicts first, per viewer byte checks from day one, and a smoke test first network saved the rest of the rework. Fixing the ladder before the sweeps kept nulls as nulls and unwired plans failing fast. Detail goes to Chapter 8.

Chapter 5: Implementation

5.1 Registry as single source of truth

One file controls the whole build. That file is `registry.json`. It lists the exact code version, the Docker image, the start command and the port for each program. There is no second source. If the registry says it, the runner does it. Full build commands are in Appendix A.

Table 5.1 gives the pins used for the draft-18 runs. I use commit hashes, not floating tags. A tag like `:latest` can move overnight and silently change what was tested.

Table 5.1: Pinned builds. One row is one program that can be rebuilt exactly.

Program	Code pin	Image	Port
moq-rs relay, pub, sub	52b8a90 (d18), 2d16c9c8 (d16), 37e636e (d14)	moq-rs/relay:14,16,18	4443
moxygen relay, FLV clients	4095b18 plus proxygen 3ad19ac	moxygen-client,relay:latest	9448
imquic relay, pub, sub	imquic 1f4cbf8 plus picoquic 0dbd19d	imquic/relay:16,17,18,19	4443
LL-DASH H2/H3 origin, client	nginx/caddy plus client.py	lldash-origin/client-h2/h3	443

A few points need plain words. moq-rs builds one commit per draft, so d14, d16 and d18 are really different code. moxygen builds once from source and the same image is retagged as :14, :16 and :18, so its draft label is a claim only. imquic also builds once and picks the draft at run time with `-M`. The relay runs with `-M any` so it talks to all drafts. A pinned relay would only offer draft 18 and would reject moq-rs peers. That reject is a strict demo tool, not proof the protocols clash. Each run still declares one draft through its subscriber flag. Chapter 6 quotes what was actually agreed.

Start commands live next to the pins in the registry. moq-rs starts `moq-relay-ietf` with cert, key and log folders, then `moq-pub` and `moq-sub` with the stream name. moxygen starts `moqrelayserver` on `/moq`, then the FLV streamer and receiver with the FLV file and namespace. imquic starts `imquic-moq-relay` with cert and qlog folders, then pub and sub with relay, port, draft and namespace. The imquic pub is a clock that sends one stamp per second. It has no video in it, so I use it for relay checks only, not for media scores. Four other builds (moqtail, moq-dev, quiche-moq, moq-go) stay disabled as unverified. That means not tested, not failed.

5.1.1 How draft agreement is checked

ALPN is proof on the wire. BOOT is only proof the box started. This split matters because a started box may still speak the wrong draft.

When two boxes shake hands, the client offers a short word that names the draft, for example `moqt-18`. The server picks one word back. My probe code reads that word from the capture and maps it to a draft. If the word is seen, the draft counts as agreed. If only the container log says it started, with no word seen, it counts as BOOT, which means unverified. An unverified claim is never used as a result. The probe returns empty rather than guess.

```
ALPN moqt-18 seen on wire  -> CONFIRMED
TAP14 setup-only ok        -> CONFIRMED
container up, no proof     -> BOOT, UNVERIFIED
build missing              -> NOT BUILT
```

In practice this gives three clear cases. moq-rs is honest but quiet on d16 and d18, its logs do not print the draft, so only the wire word confirms it. imquic confirms through a small setup only test when pinned with `-M`, for example `-M 18`. moxygen stays at BOOT. Its handshake uses `h3`, since WebTransport runs over HTTP/3 [17], and in my captures its raw offer never includes draft 18, so the d18 label cannot be trusted. Chapter 6 uses only the confirmed rows.

5.2 Topology descriptors

Each network is drawn in a small Python file with Containernet [18, 19] and Open vSwitch [20], not clicked together by hand. That way the same file gives the same network twice. See Fig. 4.2 in Chapter 4 for the picture. I do not redraw it here.

Basic is the full example. Publisher, relay and subscriber meet at one switch. That is it. Chain adds a second relay and a second switch, so the path is publisher to relay A to relay B to subscriber. The middle wire can carry its own impairment. Fanout keeps one relay but adds subscribers, so one publisher feeds N viewers. Chainfanout is fanout hung off relay B. It ran once as a trial.

Clean links are fast, 1000M with 1ms delay. Only the named wires get bad weather. By default only viewer wires are shaped. The publisher and relay stay clean so delay is not counted twice. A plan can move the weather with `network.devices`. For example, `devices.relay_a` puts loss on the middle wire in chain. Anything not named stays clean. Every run saves the kernel receipt as `tc_{host}.log`. Each run file also states its topology in plain text. Without that line a chain plan would quietly run as basic, so the check fails fast instead. Queue size applies to publisher and viewer wires only. The middle wire has no queue cap, so chain queue rows read as end to end buffer, not one shared queue.

5.2.1 Where weather goes, with an example

I copy the snippet here instead of a screenshot, so it stays sharp in print. The left block is the plan. The right block is the receipt the kernel wrote.

```
plan: network.devices:     receipt tc_relay_a.log:
  relay_a: {loss: 1}         htb shaped + netem loss 1%
sub stays clean here       sub shows no shaping
```

For fanout the plan lists each viewer on its own, like `sub1`, `sub2`, `sub3`. Each gets its own shaper. That means separate home links, not one shared queue. The shared queue test is different. It shapes the relay wire itself, which every viewer crosses. True single queue shaping would need switch level queues, so that stays future work.

5.3 Harness modules

Five small programs do the work. See Fig. 4.1 for the flow. Each does one job.

The runner builds the network, starts relays first with short gaps, then publishers once the namespace shows up, then subscribers last. It watches slow joins and restarts a stuck subscriber up to twice. It stops on delivery done or on time limit. The verifier reads relay logs and checks byte value by hash plus prefix match, not just byte count. The playable helper cleans output files and rebuilds video where the format allows it. The report builder turns the JSON summaries into HTML and CSV tables. A probe checks draft support before the network starts, so a boot only claim never counts as agreed.

Shaping is checked, not assumed. After the network starts, each host dumps its queue rules to `tc_{host}.log`. Impaired hosts must show shaping. Clean hosts must not. Missing tools or missing rules stop the run.

5.3.1 How scoring and timing work

Delivery, speed and timing each come from a different log. Mixing them up is the most common mistake, so I keep them apart here.

Delivery means byte value, not byte count. The verifier builds what the file should look like, start plus one piece per video chunk, then hashes what arrived. Full and exact is a pass. Full but wrong bytes is a mismatch and fails when integrity is on. Short but correct start is partial. Each viewer is scored alone and the worst viewer sets the run verdict.

Speed means delivered bytes times 8 divided by run time. The catch is the clock stops at different points. MoQ pushes until quiet, so it often stops early. DASH fetches step by step until the time limit, so it runs long. Both speeds are true, but they use different clocks. Every speed cell must state the clip, the stop reason, and the label. For fanout I cite file bytes only, never the relay counter, because the counter counts twice when a viewer restarts.

Timing means round trip from qlog only, per viewer. Wire timing stays empty because decrypt is off and captures hold headers only. That is honest. H2 and H3 rows have no round trip by design, so I use finish time plus the `via h3` tag instead. First object time stays empty through all runs for the same clock reason, and Chapter 6 shows it as null.

What each log holds, in one line each: pcap holds headers only, about 86 bytes, good for counts and the ALPN word. mlog holds JSON lines per link, good for objects made and hashes. The output folder holds the saved file, good for final bytes and verdict. The receipt folder holds the shaping proof.

The bottom row is the receipt drawer. It is one wide box across the page. Each host writes one small file after the network starts, named `tc_pub.log`, `tc_relay_a.log` or `tc_sub.log`. A shaped wire must show shaping words like `htb` plus `netem loss 1%` in its file. A clean wire must show no shaping in its file. The thick wire in the drawing is the shaped one. Thin wires have proof of clean instead. If proof is missing the run stops rather than passing quietly.

For fanout I cite the saved file, not the relay counter. The counter counts twice when a viewer restarts, so it can read 2x to 4x high. The file holds what that viewer really kept. Worst viewer sets the verdict.

5.4 Test plans and media

A test plan is a short YAML file. It lists which publisher, relay and subscriber to run, on which topology, with what weather, for how long, and what counts as enough. Defaults are simple: basic topology, 15 seconds, clean links. A plan only changes what it needs to.

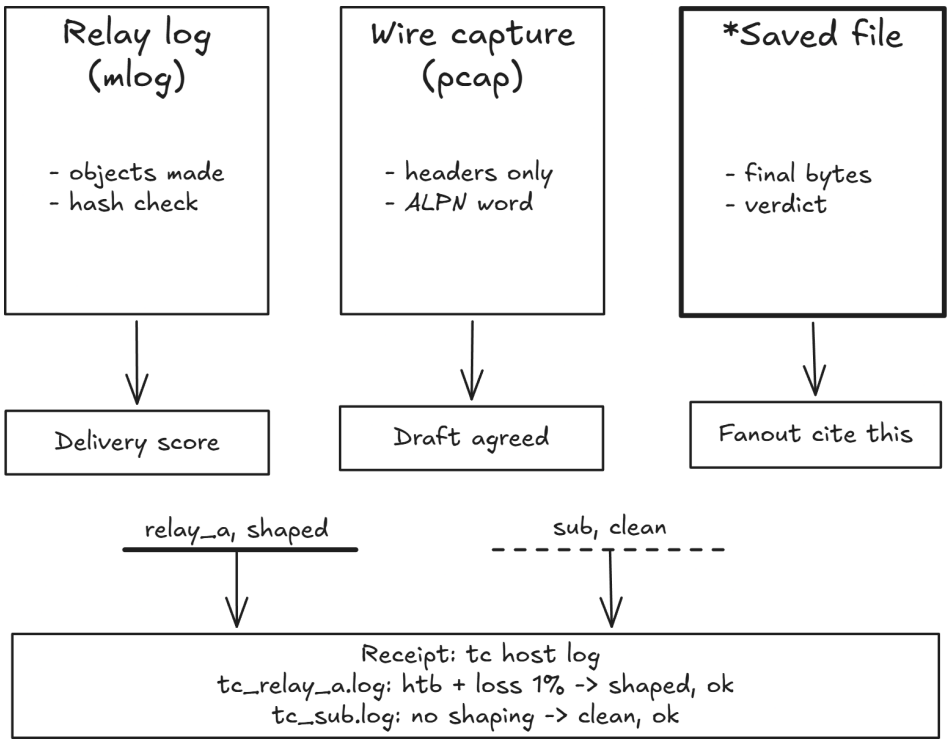


Figure 5.1: Where each score comes from. Top boxes give delivery, draft proof and final bytes. The wide bottom box is the shaping receipt.

Video is fixed too, so the protocols carry the same load. The short clip is 10 seconds with a visible timer. Good for fast checks. The long film is 120 seconds. Both use a 2 second group size, so one MoQ group lines up with one DASH segment of 2 seconds. That keeps the comparison fair. DASH files are cut with the same 2 second size with MP4Box [21] and ffmpeg [22]. Origins are nginx for H2 [23] and caddy for H3 [24, 25]. A tiny one segment file is used for honest catch up tests. FLV copies exist for moxygen, since it only speaks FLV. A Blender sample is kept for pilot runs only.

Seven future plans sit in a quarantine folder. The runner ignores them. No jitter, reorder or trace replay is wired in. Those stay future work.

5.5 Known build-time limits

These limits were known before measuring. They shape what Chapter 6 can claim.

Chain needs a shared coordinator file to link the relays. Early runs missed it and scored zero, so the fix is part of the build. Cross impl chain still fails by design since the relays never register with each other. Packet captures save headers only. Decrypt is off, so wire round trip stays null and qlog time is used instead. moq-rs needs fragmented MP4 and rejects plain Blender files without remux. moxygen needs FLV and its byte compare is close, never exact. Tags can drift from Hub, so the runner checks image IDs and stops on mismatch. What these limits did to the numbers is in Chapters 6 and 8, not here.

Chapter 6: Evaluation

6.1 Verification

RQ1 asks what each build really speaks, not what it claims. A claim counts only with wire proof. Boot only means the box started and proves nothing. Table 6.1 is the rule, Table 6.2 is what was seen.

Table 6.1: Draft tiers. Only the top row counts as agreed.

Tier	Meaning
CONFIRMED	ALPN word seen on wire, or setup only test passed
UNVERIFIED	box started, no draft proof
NOT BUILT	build missing or probe failed

Table 6.2: What each build showed. See Chapter 5 for how the probe works.

Build	Claim	Result
moq-rs 14, 16, 18	14 to 18	CONFIRMED, ALPN word seen on wire, logs silent on 16 and 18
imquic 16, 17, 18, 19	16 to 19	CONFIRMED, setup only passed when pinned with -M
moxygen 14, 16, 18	14 to 18	BOOT only, handshake hides behind h3 and raw offer never has 18, same code in all three tags
LL-DASH H2, H3	none, not MoQT	BOOT, no draft to agree since it speaks HTTP not MoQT

Two notes keep this honest. imquic with default `any` picks the highest common draft on its own, so a d18 row can agree 19 in silence. Pinning with `-M` keeps the row honest. moxygen tags :14, :16 and :18 hold the same code, so the d18 label cannot be trusted and stays at BOOT. LL-DASH rows read BOOT since they speak HTTP not MoQT, so no draft word exists to agree. I ran this check with `make verify`, which runs `python3 -m harness.verify` before any network starts. As a control, four mismatch rows (d14 pub to d18 relay and back) all failed with zero bytes and mapped to true negatives. First object time is null in all runs, so no join time claim is made.

6.2 Interop matrix

RQ2 asks which trios pass and where they fail. One ladder scores all cells, from Section 4.3. Pass means full cover. Short but correct means partial. Full but wrong bytes with integrity on means fail. Table 6.3 shows the citable smoke run with three repeats. The rest are single runs and sit in the appendix.

Table 6.3: Smoke trios, n equals 3, tiny one segment clip, integrity check off. Pass here means full cover under that setting, not byte proof. All rows read mismatch on value.

Triple	Result
moq-rs pub, relay, sub, d18 clean	pass x3
moq-rs d18 loss 1 pct and 5 pct	pass x3, loss did not move this tiny clip
imquic relay or sub in the path	fail x3, zero to low bytes
moxygen relay in the path	fail x3, zero bytes
mixed moq-rs pub to imquic path	fail x3

In plain words, same build moq-rs on d18 is the only passing MoQ path. Every cross build cell fails even on a clean link. Figure 6.1 shows this pattern at a glance: one green cell against five red. Wider runs agree, one run each in Appendix B: full set 18 rows with 5 pass, 2 partial and 11 fail, all passes moq-rs d18. Cross set 28 rows with 3 pass, 3 partial and 22 fail, all passes moq-rs d18 clean or lossy. Same build d14 and d16 read partial with low cover near 0.10, so draft still matters in one code base.

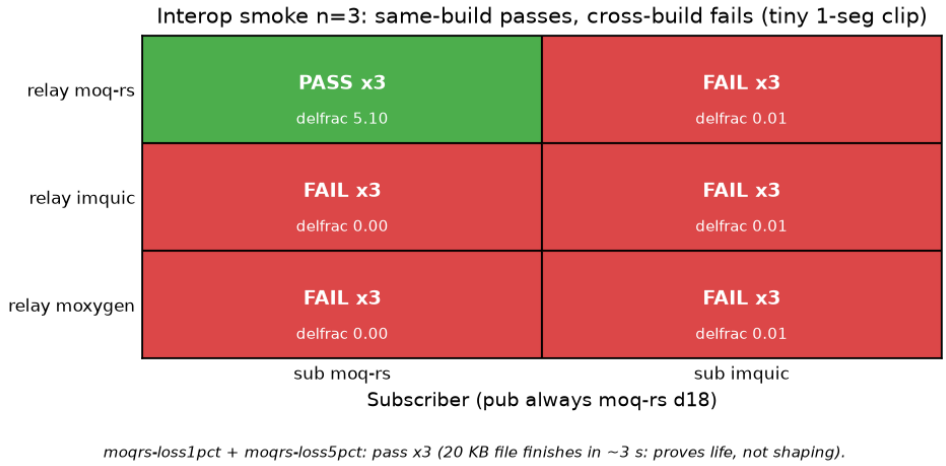


Figure 6.1: Interop smoke grid, n equals 3, pub always moq-rs d18, tiny one segment clip with integrity off. Only relay moq-rs plus sub moq-rs passes; all cross build cells fail with zero to low bytes. Pass here means full cover under smoke settings, not byte proof. Loss 1 and 5 pct rows also pass on this 20KB clip since it finishes in about 3 seconds, so they prove life not shaping. Source: `interop-quick_agg_20260908_133402.csv`.

Five causes cover the fails. One, draft word clash on ALPN. Two, path or endpoint miss on WebTransport. Three, FLV framing against fragmented MP4. Four, slow join race with restarts. Five, no backfill of missed groups for late joins. The tiny clip matters here. Loss 1 to 5 pct cannot move a 20KB file that finishes in about 3 seconds, so those pass rows prove life, not shaping. Media plus stop reason is stated with each row for that reason.

6.3 Performance

RQ3 to RQ5 ask how delivery holds under limits and scale. Each test below states a guess, why it matters, what was seen, and why. Headline rows use three repeats from `run_20260908_172709`. Loss, fanout, fairness and scale rows reached three repeats and are cited with verdicts; delay, bandwidth, queue and pilot rows use one run each and are marked exploratory. They belong in the appendix until repeated.

E1: delivery and integrity baseline

Guess: MoQ delivers full files but may fail byte proof at 120 seconds. Why it matters: cover without value would pass wrong bytes. Seen: H2 and H3 pass with exact per segment proof on all rows. MoQ on the 120 second film delivers near full cover near 0.998 on all rows, with loss 1 pct passing byte proof three times and clean passing once in three. The other 120 second rows read fail on mismatch at high cover. The 10 second control passes with proof three times. Why: scale now blocks exactness, not cover. One missed group in 61 fails the hash even at near full bytes. Clean failing while lossy passes points to a tail flake, not a transport wall.

E2: speed with stated clocks

Guess: MoQ reads faster but on a different clock. Why it matters: mixing clocks fakes a win. Seen: MoQ clean near 2.33M against H2 clean near 0.87M and H3 clean near 1.19M. Loss 1 pct and bw 5M rows show the same order, about 2x to 3x above baselines on file bytes. Why: MoQ pushes until quiet in about 3 seconds while DASH fetches paced to the 40 second limit. Both speeds are true on their own clock, so each cell states clip plus stop reason plus via tag. Old 200x gap files are not cited. That gap was a delivery bug at an old pin, fixed by repin plus join gate plus supervision.

E3: loss, delay, bandwidth and queue sweeps

Guess: single viewer MoQ survives wide limits, fanout degrades first. Why it matters: claims must name which wire was shaped. Seen on moq-rs basic at one run each: loss 0 to 20 pct holds near 14M to 29M on the short clip, delay 5 to 200ms stayed flat across single runs, bandwidth 1 to 100M stayed flat since the clip is far below cap, queue 10 to 1000 stayed flat. Figure 6.2 makes the flatness visual for delay and bandwidth on the short clip. The loss sweep re-ran to n equals 3 did not confirm the hold: loss 0.1 pct reads partial thrice near 0.14 cover while loss 5 pct passes thrice, and 0, 2, 10 and 20 pct split across repeats (`loss-depth_agg_20260910_120253_n3.csv`). Lower loss failing where higher loss passes reads as tail flake at the relay ceiling, not transport, so the loss sweep stays appendix only and its variance is reported as evidence rather than promoted. Fanout loss rows fall first at both n equals 1 and n equals 3, with 5 pct loss partial thrice and 1 pct loss mixed. Why: QUIC hides loss for one viewer on a tiny file, while per viewer shapers plus restart counts split fanout early. Delay, bandwidth and queue rows are n equals 1 with shaping receipts, so they are appendix only. No jitter, burst or trace replay was wired in.

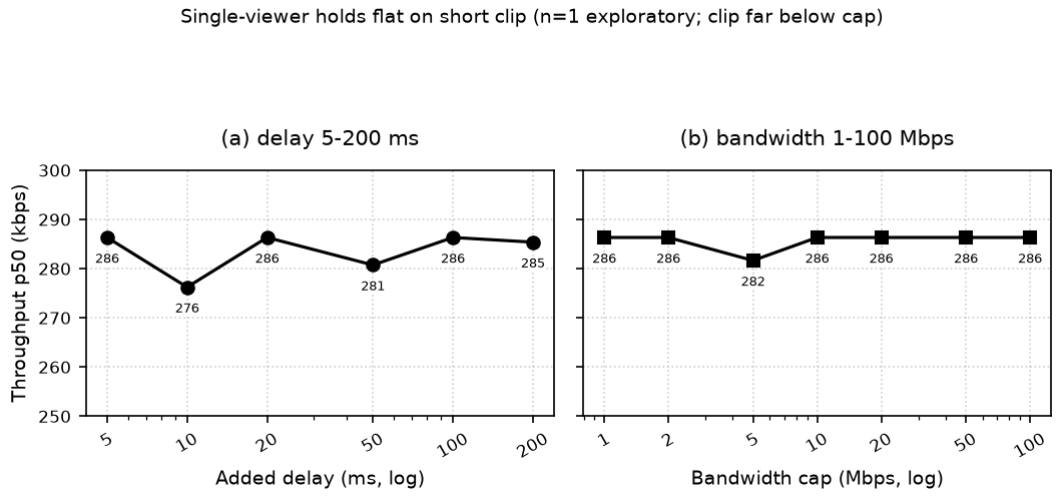


Figure 6.2: Single viewer sweeps on moq-rs basic, short `sample.mp4` clip, n equals 1 exploratory. Delay 5 to 200ms held near 276k to 286k and bandwidth 1 to 100M held near 282k to 286k since the clip sits far below cap, so these rows show life not shaping. Bars are `throughput_p50`; each row carries its `tc_{host}.log` receipt. Source: `latency_agg_20260908_150236.csv` and `bandwidth_agg_20260908_150236.csv`.

E4: fanout per viewer and fairness

Guess: one slow viewer should set the verdict. Why it matters: scale claims need each viewer scored. Seen at n equals 3: moq-rs d18 serves exactly one viewer per row and starves the

rest, at every fanout size tested. Per subscriber tables read `[full 1/2]`, `[full 1/3]`, `[full 1/5]` and `[full 1/8]` with 0 bytes on the rest, clean and lossy, on the short clip (`fanout-depth_agg_20260910_120253_n3.csv`, `fanout-n5n8_agg_20260910_113129_n3.csv`). Under the worst viewer rule these rows read fail, and a fixed worst subscriber referee reproduces that verdict at n equals 3 (`fanout-verify_agg_20260910_201632_n3.csv`: fail thrice; two repeats show winner plus starved viewer while the third transmitted the full stream on the loser's connection yet wrote zero bytes to disk with no client error, so the fail verdict holds on two distinct mechanisms). Relay logs accept every subscribe yet per connection logs show objects flowing to exactly one connection, with the rest acknowledged then silent and error free. The d16 pin instead shares one fate across viewers at n equals 3 (`draft16-probe_agg_20260910_201632_n3.csv`): N equals 2 on the easy clip reads `[full 2/2]` thrice with one byte proof good, and on the longer clip both viewers share one identical partial thrice with Jain 1.0 every repeat, so the single consumer behaviour is a d18 pin regression, not a protocol trait. The fourth corner closes the square on easy media at n equals 3 (`d18-1seg-fanout_agg_20260910_204241_n3.csv`: fail thrice, two classic starvations with the winner byte proved good once, plus one repeat where the relay transmitted the full stream on the loser's connection yet zero bytes reached disk). Controls behave: H3 serves 5 of 5 exact with Jain 1.0 at n equals 3, and imquic clock fanout serves 3 of 3 with Jain near 1.0 at n equals 3, proving the harness topology and scoring carry N viewers and isolating the moq-rs d18 relay. Mixed and moxygen fanout fail with zero bytes throughout. Late join at 5 seconds reads partial, which is correct live sense since missed groups never fill back. Why: per viewer files plus worst sets verdict stops one healthy viewer hiding starving ones, and the file bytes are cited, never relay speed, since restarts inflate counters 2x to 4x with explicit mlog against artifact warnings. Fairness index stays citable for H2 and H3 at 1.0 on three repeats, now including H3 at N equals 5. MoQ fairness is not computable with fewer than two positive viewers per repeat and stays exploratory.

E5: H2 and H3 three-way

Guess: H3 beats H2 on transport, MoQ against H3 tests the app. Why it matters: without H3 the MoQ gain could be QUIC alone. Seen in Table 6.4 and Figure 6.3: H3 clean near 1.19M against H2 clean near 0.87M, about 37 pct transport gain. MoQ clean near 2.33M sits about 1.95x above H3 and 2.67x above H2 on the same 120 second film. Loss 1 pct and bw 5M rows keep the same order. Why: H2 against H3 splits transport, MoQ against H3 splits app push against paced fetch, both on QUIC. Round trip is qlog only for MoQ and null for H2 and H3 by design, so transport is read from finish time plus via tag, never from mixed clocks.

Table 6.4: Headline three-way (MoQ vs H2 vs H3), n equals 3, 120 second film unless 10s is named. MoQ stops on quiet, DASH stops on time limit. First object null throughout.

Row	Bytes and proof	Read
h2-clean, h2-bw5, h2-loss1	pass x3, exact x3, time limit, via h2	paced fetch baseline
h3-clean, h3-bw5, h3-loss1	pass x3, exact x3, time limit, via h3	QUIC transport gain over H2
moq-clean	fail, pass, fail with good once, quiet stop, via moq	full cover, value flake at 60 of 61 once
moq-loss1	pass x3, good x3, quiet stop, via moq	full cover and proof, best MoQ row
moq-bw5	fail x3, mismatch x3, quiet stop, via moq	full cover, value miss
moq-clean-10s — h2-10s — h3-10s	pass x3, good or exact, quiet stop MoQ, time limit DASH	small clip clean for all

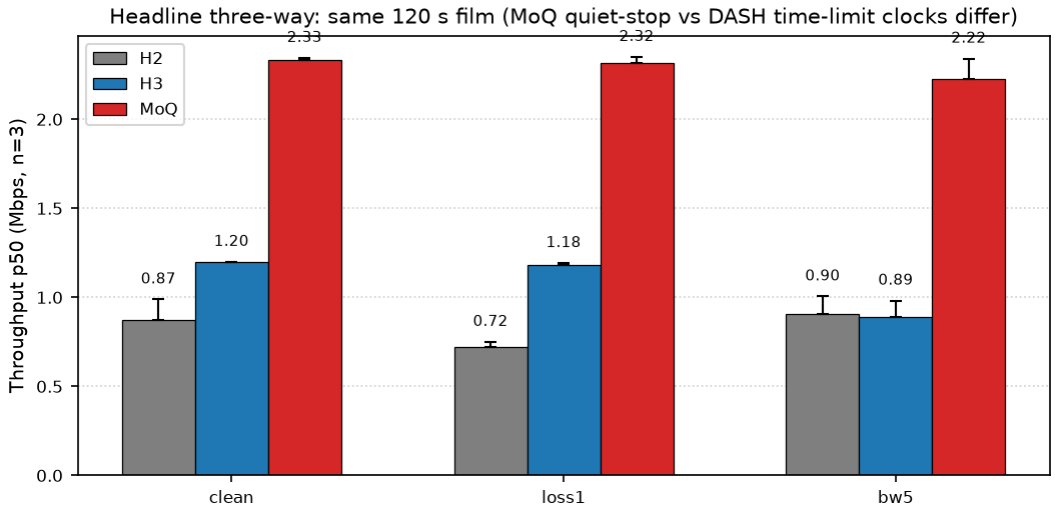


Figure 6.3: Headline three-way throughput p50 in Mbps, n equals 3, same 120 second film with 2 second groups. H3 clean near 1.20M beats H2 clean near 0.87M by about 37 pct (transport gain); MoQ clean near 2.33M sits about 1.95x above H3 (app push against paced fetch, both on QUIC). Error caps run to p95. MoQ stops on quiet in about 3 seconds while DASH fetches paced to the 40 second limit, so each bar uses its own clock and reads with `completion_reason` plus `via` tag in Table 6.4. First object null throughout. Source: `moq-vs-lldash_agg_20260908_172709.csv`.

6.4 Chain status

Relay chaining is exploratory, not a result, one run each in Appendix B. Same build moq-rs chain passes 2 of 6 on the short clip, with loss 1 pct on the middle wire also passing. Cross build chain and all chain fanout rows fail, 0 of 5, with near zero bytes. The cause sits in the demo coordinator that links the relays, not in the shaped network. Early runs missed the shared file and scored zero. With the fix the same build path passes. Cross builds never register with each other by design. Middle wire claims shape `devices.relay_a` with `tc_relay_a.log` proof, or the middle stayed clean. Full chain work goes to Chapter 8 as future work.

6.5 Chapter summary

RQ1 is answered in 6.1 with tiers and controls. RQ2 is answered in 6.2 with the tuple table and five fail causes. RQ3 to RQ5 are answered in 6.3 with E1 to E5, headline n equals 3 citable and loss, fanout, fairness and scale at n equals 3 with delay, bandwidth, queue and pilots appendix until repeated. Chain scope is fixed in 6.4. Bounds and future items go to Chapters 8 and 9.

Chapter 7: How effective is AI for these tasks

Early implementation and debugging used Deepseek V4 Flash (free tier); later work in September used Muse Spark 1.2; the one explanatory figure in Chapter 2 was generated by Gemini from text pointers I supplied and carry no measurement data. What follows judges the models on networking work only: where they helped, where they failed, how every output was checked, and where they should not be used again.

7.1 Where AI was used

AI wrote first drafts of Dockerfiles, test-plan files, small parsers, regular expressions, certificate helpers and report scaffolding, and, when given an explicit per-folder recipe, summarised run logs into per-implementation failure accounts (relay faults versus subscriber output artefacts). Protocol logic, impairment placement, verdict semantics, bug ordering and reproducibility design stayed human tasks, with AI as a sparring partner at most. Figure captions in Chapter 2 mark the Gemini images as schematics, not results.

7.2 Performance by task

Boilerplate and scaffolding were strong: fast drafts with error handling when asked, needing only efficiency work such as layer caching and disk budgeting. All of the above helped, but there were issues in Dockerfiles. I am a little familiar with Docker but AI was helpful in creating these Dockerfiles, with error handling as well when asked for. The first draft for Dockerfiles was bad, and I wasted a lot of time. If not for my knowledge of builder cache, I would not have known to give such a prompt to save time. Directed log triage was useful when directed: with the recipe fixed, models correctly separated relay truth (timeouts, bad flags, missing certificates) from subscriber output fragments counted as delivered bytes, but unprompted they merely confirmed the fix already named. AI can be helpful when giving it logs. It helped me prepare smoke tests and confirm if the network works or not, as it does good work when given explicit instructions. Protocol and impairment reasoning was weak: version negotiation versus claimed support, object delivery semantics, relay chaining, shaping placement and audit discipline all needed correction, as did verdict design (value versus count, partial versus fail, worst subscriber rule). AI just read the readme and did not verify the commands. I wasted more time in building and testing images. It just created basic tests. It did not know what topologies to use and had to take input from me before it got everything right. Prioritisation and resource planning should not be delegated: under interacting failures the models pushed the locally shiny next test while the global causal chain (no real byte has arrived, so larger media proves nothing) and the build budget stayed human calls. AI got confused again about what I asked it to do. It got sidetracked because of unsolved bugs and was not able to fix one issue properly. I guided it to follow my given order.

7.3 How AI output was verified

Nothing AI wrote became evidence without a human-run gate: every output was checked against log bytes and a rerun scored under the single verdict rule of Section 4.3, with nulls kept as nulls and unwired plans failing fast rather than passing silently. My clearest exhibit

is the stdout lie. Output files read 200, 6753 and 0 bytes that looked plausible but held log fragments and banners, with zero bytes of real video in the whole batch. Logs held the truth and output held the lie, so only gated artefacts entered Chapters 5 and 6. This loop is the independence claim of this chapter: AI proposed, the testbed disposed, and only gated artefacts entered Chapters 5 and 6.

7.4 Lessons for future work

Use AI for directed synthesis and scaffolding; keep protocol logic, impairment math, ordering and reproducibility design human. Concretely: decompose triage before delegating it, budget builds before starting them, and never accept a next-step proposal that the current log bytes contradict.

Chapter 8: Discussion

8.1 Findings and meaning

I present no new measurements in this chapter. What follows is my judgment on the Chapter 6 results: which rows I trust, what I think they mean, and what I would report upstream. I only treat rows with three repeats as citable. The headline three-way clears that bar, and so do the loss, fanout and fairness reruns from 10th September. Everything run once, delay, bandwidth, queue, pilots and chain, stays in the appendix until it is repeated.

The pattern I kept hitting was a ceiling on exactness rather than cover. On the 120 second film MoQ delivered near-full cover, near 0.998, yet failed byte proof on most rows, because a single missed group out of 61 is enough to fail the hash. Strangely the clean row failed while lossy rows passed, which is why I read this as a tail flake at the relay ceiling rather than a transport limit.

The single-viewer sweeps held for delay, bandwidth and queue, but the loss sweep did not survive its three-repeat rerun: 0.1 pct loss failed three times while 5 pct passed thrice and 0, 2, 10 and 20 split. Lower loss failing where higher loss passes cannot be transport behaviour, so I report the loss sweep as variance evidence and leave it appendix-only. Fanout loss broke first, falling to partial three times at 5 pct. My reading is that QUIC hides loss for one viewer on a tiny clip, while per-viewer shapers split fanout early.

The fanout result surprised me, so I checked it hardest. Under the worst-viewer rule moqrs d18 serves exactly one viewer and starves the rest at every size from 2 to 8, with per-connection logs showing every subscribe accepted yet objects flowing to one connection only, confirmed by dedicated 25 second full-window rows at n equals 3 (`fanout-verify_agg_20260910_201632_n3.csv`). The d16 pin instead shares one fate across viewers at n equals 3 (`draft16-probe_agg_20260910_201632_n3.csv`), and H3 and the imquic clock fanout serve everyone, so I report this as a d18 pin regression rather than a property of the protocol. I cite file bytes throughout, never the relay counters, since restarts inflate them 2x to 4x. Fairness stays citable for H2 and H3 at 1.0; MoQ fairness cannot be computed with fewer than two healthy viewers per repeat.

For the working group I would report the d18 single-consumer gap with the d16 contrast, plus group skip against restart inflation, joins with no retry, flush loss on close, the near-zero cross-build partials, and the cover and prefix thresholds. The remaining pilots are subgroup drop, which ran once, and datagram against stream, which needs code since the mode key is still unwired. Loss knee, late-join cache fill, pinned against any, FLV parity, queue against loss and leg split all reached three repeats and sit in Appendix B with their verdicts.

8.2 What held, what did not, and what was left out

What I am confident about is the method rather than any single number. One verdict rule scored every cell, byte proof by hash plus prefix with the worst viewer setting the verdict, and every weather claim carries its shaping receipt. The same-source three-way is what lets me split the H3 transport gain, about 37 pct over H2, from the application gain above it. What did not work is equally clear. The long-file ceiling held, with near-full cover near 0.998 but

byte proof failing on most 120 second rows, the exceptions being loss 1 pct thrice and one clean pass in three. Fanout scale failed past one viewer on the d18 pin at every size from 2 to 8. Chain stayed exploratory at 2 of 6 same build and 0 of 5 chainfanout, with cross builds failing by design. Captures stayed header-only with decrypt off, so wire time stayed null and qlog time stayed canonical. I report all four as bounds, not footnotes.

I left several things out, each for a reason I can name. Shared-queue tails need switch-level queues while my shaping sits at host edge with burst unset. Rate logic and stall scores need a switching policy while the harness sets static limits only. In-runner repeats, wire and first-object curves, and tag-drift guarding each need machinery that time did not allow: five repeats for tight bounds would cost under one VM hour, clock discipline with keylog decrypt, and image-ID checks. WebRTC was a deliberate exclusion, since call servers do not compare with cache-style file fetch on one link model. Trace replay, burst, jitter and reorder were scope choices rather than oversights, and quarantined plan keys fail fast instead of running clean silently. Chain stays exploratory for the coordinator reasons given above, and the large-file video stays pilot pending a relay fix with video-only remux plus a rebuild check.

8.3 If I started today

I began this project with no idea how research is done, what to read, what to frame as questions, or why. That cost me more than any bug. Builds and pins ate weeks, but not knowing where to start ate months. If I started today I would work in this order: survey and papers before touching code, since I chased BBB tracks early, hit a known bug, and abandoned it with too little time left. Only then would I use the implementations hands-on, and build Docker images once I knew which core features had to be inside. Verdicts and quarantine would come before any sweep, with per-viewer checks from day one so one healthy viewer cannot hide starving ones, and a smoke network with receipts before any sweep so nulls stay nulls. I would budget builds as summed per-run floors under one-by-one runs, freeze plans during runs, and write while the VM runs. Impairments and topologies would grow slowly with interop last, because my supervisor pushed QUIC-versus-TCP logic over copying prior tests blindly: am I running a test because prior work did, or because it probes something custom topologies reveal about QUIC and MoQT. Each of these ties to a Chapter 4 lesson. Method before measurement is the line I would pin above my desk.

Chapter 9: Conclusion

This work built a reproducible testbed for Media over QUIC Transport on Containernet and Docker. It pinned moq-rs, imquic and moxygen plus LL-DASH over HTTP/2 (H2) and HTTP/3 (H3), all playing the same film with fixed groups. Loss, bandwidth, delay and queue were set per link with kernel receipts under static limits only. Each viewer was scored on byte value with strict tiers, and the worst viewer set the verdict. That method is the main result, since it lets others rerun and recheck each claim.

RQ1 found moq-rs confirmed on the ALPN word plus imquic confirmed when pinned, and moxygen at boot only (6.1). RQ2 found same build d18 the only pass path, with cross builds failing even when clean across five known causes (6.2). RQ3 found H3 about 37 pct above H2, with MoQ near 2.3M about 1.95x H3 on the same film but with value misses on most long rows (6.3). RQ4 found single viewer holds across delay, bandwidth and queue while the loss sweep scattered partials at three repeats, with delay, bandwidth, queue and pilot rows appendix only (6.3). RQ5 found moq-rs d18 fanout serves one viewer and starves the rest at every size from 2 to 8 while H3 and imquic clock fanout serve all, with H2 and H3 fair at 1.0 and MoQ fairness exploratory (6.3). RQ6 lists the d18 single consumer relay gap with the d16 shared fate contrast, plus group skip, join retry, flush loss and threshold sense for the working group (see Findings in Chapter 8). Join cost and wire time stayed null throughout and transport was read from finish time plus via tags.

Three follow ups remain, detailed in the next section. The testbed as it stands is ready to carry those runs and to feed auditable reports back to the working group.

On MoQT itself, the direction looks sound but the product is not there yet: relay caching with deadline-skipping over QUIC gives a credible path to sub-second delivery at CDN-like scale on one `moqt://` stack, as draft-20/21 sets out [2], yet this study finds only same-build paths passing, byte proof flaking on long files, and fanout regressing past one viewer on the d18 pin. The goals are therefore realistically achievable if interop hardens, ABR and monitoring land, and relays prove shared-fate fanout under repeats — all of which this testbed can now recheck.

9.1 Future work

First, finish the remaining one run rows (delay, bandwidth and queue sweeps plus the sub-group drop pilots) to three repeats so they become citable bounds with error bars, and file the d18 single consumer fanout gap upstream with the per connection evidence. Second, add clock control plus key log decrypt for true first object and wire curves. Third, fix the relay for large file video with video only remux, and thread the mode key for a datagram against stream pair. Loss, fanout, fairness and scale rows already reached three repeats and stay citable as stated. All three reuse the pins, plans and receipts in Appendix A with no rebuild of the harness.

Appendix A: Build and evidence index

A.1 Build pins and commands

One file runs it all. That file is `registry.json`. I pinned moq-rs per draft at 52b8a90 for d18, 2d16c9c8 for d16 and 37e636e for d14. I built moxygen once at 4095b18 plus proxygen 3ad19ac and retagged that one image to :14, :16 and :18. I built imquic once at 1f4cbf8 plus picoquic 0dbd19d and it picks the draft at run time with `-M`. Ports are 4443 for moq-rs and imquic, 9448 plus `/moq` for moxygen, and 443 for the H2 and H3 origins. Start commands and cert names sit next to the pins in the same file. The runner checks image IDs before runs and I retag by script before headline rows. If the check fails the run stops.

A.2 Evidence index

I keep it simple here. The headline three-way aggregate holds 16 rows at 3 repeats, built from the `run_20260908_1727` batches. The smoke interop aggregate holds 8 rows at 3 repeats. Loss, fanout and fairness reached 3 repeats on 10th September (`loss-depth_agg_20260910_120253_n3.csv`, `fanout-depth_agg_20260910_120253_n3.csv`, `fairness-depth_agg_20260910_120253_n3.csv`). The fanout scale extension holds 4 rows at 3 repeats (`fanout-n5n8_agg_20260910_113129_n3.csv`: moq-rs 5sub, 8sub and 5sub loss1 plus H3 5sub), the draft 16 pin probe holds 3 rows at 3 repeats (`draft16-probe_agg_20260910_201632_n3.csv`), the d18 fourth corner holds 1 row at 3 repeats (`d18-1seg-fanout_agg_20260910_204241_n3.csv`), and the full window referee check holds 1 row at 3 repeats (`fanout-verify_agg_20260910_201632_n3.csv`, batches `run_20260910_172052` plus two phase 8 repeats). Full file names sit with the pasted blocks in the last appendix. Delay, bandwidth, queue and pilot rows are one run each and live in this appendix only. Each run keeps `logs/`, `qlogs/*.mlog`, `tc_{host}.log`, output files and `segments.json` with via tags. The quarantined future plans stay ignored by the runner. Phase 7 runs must go via sudo since Containernet prints "Mininet must run as root" otherwise; the 20260910 run without sudo failed loss, fanout and fairness and only the sudo wired fanout-n5n8 stage produced batches. Statuses on multi subscriber MoQ rows from before the worst subscriber referee fix read pass on the sum; the per subscriber tables underneath them are the citable record and read worst viewer.

A.3 Commands I ran

I list each command with what it does and how I used it.

Build first, one per program. `make build-moq-rs`, `make build-moxygen`, `make build-imquic`, plus `make build-lldash-h2` and `make build-lldash-h3` for the baselines. Each builds the Docker images at the pins in Table 5.1.

Check drafts before any network. `makeverify` runs `python3-mharness.verify`. It probes each claimed draft and prints CONFIRMED, BOOT or NOT BUILT. BOOT never counts as agreed.

```
make verify
python3 -m harness.verify
```

Run one plan with one worker. Workers stay at 1 so runs go one by one and do not fight over the same ports.

```
python3 run_testbed.py --plan testplans/moq-vs-lldash.yaml --workers 1
python3 run_testbed.py --plan testplans/interop-quick.yaml
  --workers 1 --verify
```

The `--verify` flag runs the draft check first, then the network. I used it on smoke runs.

Stage big files on local disk. Shared folders are slow for captures, so `MOQ_FAST_IO=1` stages output, qlogs and pcaps on local disk during the run and copies them back before scoring. Without it the run is unstaged. In the Makefile this is the `FAST_IO` knob.

```
MOQ_FAST_IO=1 python3 run_testbed.py
  --plan testplans/bbb-smoke.yaml --workers 1
make test-bbb-smoke FAST_IO=1
```

Repeat the headline three times. The phase script loops the plan and numbers each batch. `REPEATS_HEADLINE=3` is the default. One pass is only for a fast smoke check.

```
REPEATS_HEADLINE=3 ./evaluation_tests/phase4_baselines.sh
REPEATS_HEADLINE=1 ./evaluation_tests/phase4_baselines.sh
```

Merge repeats into one table. The aggregate script takes batch folders and writes one CSV. `--min-repeats 3` keeps only rows with three repeats. I used `--min-repeats 1` for single run pilots, which stay appendix only.

```
python3 analysis/aggregate.py artifacts/runs/run_20260908_172710
  artifacts/runs/run_20260908_172915 artifacts/runs/run_20260908_173116
  --out artifacts/reports/moq-vs-lldash_agg_20260908_172709.csv
  --min-repeats 3
make report
```

Score every viewer, worst sets the verdict. The referee fix in `harness/runner.py` drives fanout status from the worst subscriber instead of the byte sum, with `worst_sub_role` recorded per row and regression tests in `tests/test_completion.py`. A full window check disables early quiesce through the now wired `network.quiet_period` key (`harness/plan_validation.py`) so a slow viewer cannot be blamed on teardown timing:

```
sudo MOQ_FAST_IO=1 python3 run_testbed.py
  --plan testplans/fanout-verify.yaml --workers 1
```

`make report` builds the HTML plus CSV tables from the run summaries. Each run folder keeps its own `report.html` next to `summary.json`.

Appendix B: Sweep and pilot tables

Delay, bandwidth, queue and pilot rows ran once each, so do not cite them yet. They need three repeats first. Loss, fanout, fairness and scale rows reached three repeats and are cited in Chapter 6 with the verdicts stated there. Each row still states the clip plus stop reason plus shaping receipt. On fanout rows I cite file bytes, never relay speed.

B.1 Loss, delay and bandwidth sweeps

Loss on basic moq-rs held from 0 to 20 pct near 14M to 29M with cover near full at one run, but the three repeat rerun scattered partials with 0.1 pct failing thrice, so I keep the sweep appendix only and cite its variance. Chain loss 1, 5 and 10 hold near 14M. Fanout loss 1 mixes pass with partials and loss 5 reads partial thrice near 0.11 to 0.19 cover. So fanout breaks first. Delay 5 to 200ms stays flat near 276k to 286k. Bandwidth 1 to 100M stays flat near 281k to 286k since my short clip sits far below cap. Those rows just show life, not shaping. Round trip here is qlog time only.

B.2 Queue and bufferbloat

Queue 10 to 1000 plus chain plus fanout all pass near 284k to 286k. One fanout queue 100 row reads low near 14k next to 285k rows. I treat that as one sample noise. Bufferbloat rows at 5M with queue 10, 50, 200 and 1000 all pass with no curve on small media. That is an honest null. Queue cap hits publisher and viewer wires only. So chain queue rows read as end to end buffer.

B.3 Fanout, fairness and chain single runs

At three repeats moq-rs d18 serves exactly one viewer per row with the rest at zero bytes: 2, 3, 5 and 8 viewers all read `[full 1/N]` with mismatch on value for the served viewer. The d16 pin probe reached three repeats (`draft16-probe_agg_20260910_201632_n3.csv`): N equals 2 on the easy clip reads `[full 2/2]` thrice with one byte proof good, and both viewers share one identical partial thrice with Jain 1.0, so d16 shares fate where d18 splits it. The d18 easy clip corner reads fail `[full 1/2]` at n equals 3 with the winner byte proved good once (`d18-1seg-fanout_agg_20260910_204241_n3.csv`). imquic to 3 passes with low byte receipts on all three viewers and holds my only MoQ fanout fair index near 1.0. H3 to 5 viewers passes exact with Jain 1.0 at n equals 3 (`fanout-n5n8_agg_20260910_113129_n3.csv`). Mixed rows fail with a wide split near 78x. moxygen fanout fails at zero. Late join at 5 seconds reads partial. That is right for live TV since missed groups never come back. Fairness rows serve one viewer each near 283k to 286k and hold no fair index since too few viewers stayed positive. Chain same build passes 2 of 6 with loss 1 on the middle wire passing. Cross build chain and all chain fanout fail at 0 of 5. That sits with the coordinator, not the network.

B.4 Pilot rows

Most pilots reached three repeats. The loss 5 knee mixes fail, pass and fail. The 30 second MoQ late join fails thrice while the H3 late join passes good thrice near Jain 1.0, so no back-fill is the rule on MoQ. Pinned against any passes thrice on imquic. FLV loss and bandwidth

parity fail thrice at zero bytes. Queue times loss passes thrice on all four rows. Middle against edge split passes thrice on both legs, good throughout on the relay leg. Subgroup drop ran once each and falls to partial near 0.16 to 0.63 under load with no fall to zero. That backs drop to latest as my pressure story. Big file video hit two thirds once by luck then stuck at the start. So it stays future work until the relay is fixed plus video only remux. Zero byte stall rows are race exhibits. Never speed points.

Appendix C: Raw evidence excerpts

No screenshots here. I pasted each block straight from my run folders so it stays sharp and you can recheck it. The pasted blocks below come from `artifacts/runs/run_20260908_173745`, one batch of the headline set, with the full table at `artifacts/reports/moq-vs-lldash_agg_20260908_172709.csv`. The `report.html` in each run folder holds the same numbers with charts.

C.1 Headline aggregate rows

Three rows from my citable table, 3 repeats each. Proof sits next to speed. Round trip is null for H2 and H3 by design. First object is null throughout.

```
run_id,n,statuses,verdicts,throughput_p50,rtt_n,ttfo_n_null
h3-clean,3,pass+pass+pass,exact+exact+exact,1195910.0,0,3
moq-clean,3,fail+pass+fail,mismatch+good+mismatch,2331016.0,3,3
moq-loss1,3,pass+pass+pass,good+good+good,2316074.0,3,3
```

C.2 Fanout scale rows

Four rows from `artifacts/reports/fanout-n5n8_agg_20260910_113129_n3.csv`, 3 repeats each on the short clip. MoQ cites file bytes only since the relay counter inflates 2x to 3x with mlog/artifact WARNs. Statuses below are runner completion; delivery reads fail under the worst viewer rule since exactly one viewer per row holds bytes ([full 1/5], [full 1/8]). H3 cites throughput with Jain 1.0.

```
run_id,n,statuses,verdicts,artifact_thr_p50,jain_p50
fanout-moqrs-5sub,3,pass+pass+pass,mismatch x3,284654.3,-
fanout-moqrs-8sub,3,pass+pass+pass,mismatch x3,284654.3,-
fanout-moqrs-5sub-loss1pct,3,pass+pass+pass,mismatch x3,281854.4,-
fanout-h3-5sub,3,pass+pass+pass,exact x3,1078918.1,1.0
```

Per subscriber tables behind these rows show one viewer near 107457 bytes and the rest at zero in all three repeats; the artifact speed above is that single viewer, not a share. The dedicated full window row `fanout-verify-2sub-fullwindow` reads fail at n equals 3 (`artifacts/reports/fanout-verify_agg_20260910_201632_n3.csv`): two repeats show winner plus starved viewer, while the third (`artifacts/runs/run_20260910_202419`) transmitted the full stream on the loser's connection per its paired log yet wrote zero bytes to disk with no client error, confirming teardown timing does not explain it on either mechanism.

Shared fate at three repeats (`artifacts/reports/draft16-probe_agg_20260910_201632_n3.csv`), with the byte proof repeat in `artifacts/runs/run_20260910_201633`:

```
run_id,n,statuses,verdicts,artifact_thr_p50,jain_p50
d16-fanout-2sub-1seg,3,pass+pass+pass,mismatch+good+mismatch,13317.6,1.0
d16-fanout-2sub-mirror,3,partial+partial+partial,mismatch x3,7106.1,1.0
d18-fanout-2sub-1seg,3,fail+fail+fail,mismatch x3,6531.3,-
```

The d18 corner rows (`artifacts/reports/d18-1seg-fanout_agg_20260910_204241_n3.csv`) read fail thrice: two classic starvations with the winner full at 20892 bytes and byte proved good once, plus one wire-full capture-zero repeat, closing the pin against media square. The lone Jain 1.0 in that file mixes artifact bytes against mlog fallback on a zero-capture viewer and is not cited as fairness.

C.3 Shaping receipt

From `moq-loss1/logs/tc_sub.log`. My viewer wire shows shaping with loss 1 pct. Clean hosts show no shaping. I stripped the control codes.

```
qdisc htb 5: dev sub-eth0 root refcnt 7
qdisc netem 10: dev sub-eth0 parent 5:1
  limit 1000 delay 10ms loss 1%
```

C.4 DASH sidecar proof

From `h3-clean/output/h3-clean.segments.json`. I fetched all 60 parts over H3 with zero fallback. So my H3 gain is transport, not mixing.

```
fetch_protos: {h3: 61}
fetched: 60 / 60  skipped: 0
{"name": "chunk-stream0-00001.m4s",
 "bytes": 15356,
 "sha256": "96441ec7...bfedd541"}
```

C.5 Relay log proof

From `moq-loss1/logs/relay.log` and `qlogs/*_server.mlog`. My relay listens, the handshake reads client setup then makes server setup, and my saved file holds 903210 bytes.

```
moq_relay_ietf::relay: listening on [::]:4443
moqt:control_message_parsed stream_id 0 client_setup
moqt:control_message_created stream_id 0 server_setup
output/moq-loss1  903210 bytes
```

Bibliography

[1] IETF, "Media Over QUIC (moq)." `https://datatracker.ietf.org/group/moq/about/`, 2026. IETF Working Group.

[2] S. Nandakumar, V. Vasiliev, I. Swett, and A. Frindell, "Media over quic transport." Internet-Draft, draft-ietf-moq-transport-20, Aug. 2026. Work in progress.

[3] H. H. Pandey, "Reproducible media-over-quic testbed." `https://github.com/Hardik-7892/moqt-testbed`, 2026. GitHub repository.

[4] ISO/IEC, "Information technology – dynamic adaptive streaming over HTTP (DASH) – part 1: Media presentation description and segment formats." ISO/IEC 23009-1, 2022.

[5] R. Pantos and W. May, "HTTP live streaming," RFC 8216, Internet Engineering Task Force, Aug. 2017.

[6] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, Internet Engineering Task Force, May 2021.

[7] E. André, N. Le Breton, A. Lemesle, L. Roux, and A. Gouaillard, "Comparative study of WebRTC open source SFUs for video conferencing," in *2018 Principles, Systems and Applications of IP Telecommunications (IPTComm)*, pp. 1–8, 2018.

[8] Z. Gurel, T. E. Civelek, D. Ugur, Y. K. Erinc, and A. C. Begen, "Media-over-QUIC Transport vs. Low-Latency DASH: a Deathmatch Testbed," in *Proceedings of the 15th ACM Multimedia Systems Conference*, MMSys '24, (New York, NY, USA), pp. 448–452, Association for Computing Machinery, Apr. 2024.

[9] ISO/IEC, "Information technology – multimedia application format (MPEG-A) – part 19: Common media application format (CMAF) for segmented media." ISO/IEC 23000-19, 2024.

[10] M. Nguyen, P. Nys, S. Pham, D. Silhavy, S. Arbanowski, and S. Steglich, "Streaming Face-Off: A Testbed Analysis of Media-over-QUIC and Low-Latency DASH," in *Proceedings of the 16th ACM Multimedia Systems Conference*, MMSys '25, (New York, NY, USA), pp. 317–321, Association for Computing Machinery, Mar. 2025.

[11] J. Kuo, "Evaluating media over quic (moq) for low-latency adaptive streaming: Latency performance and abr possibilities of moq in comparison to low latency http adaptive streaming and real time communication," degree project in computer science, second cycle, 30 credits, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, July 2025. Supervisors: Torbjörn Einarsson and Mohit Daga; Examiner: Viktoria Fodor; Host company: Eyevinn Technology.

[12] Z. Gurel, T. Erkilic Civelek, A. Bodur, S. Bilgin, D. Yeniceri, and A. C. Begen, "Media over QUIC: Initial Testing, Findings and Results," in *Proceedings of the 14th ACM Multimedia Systems Conference*, MMSys '23, (New York, NY, USA), pp. 301–306, Association for Computing Machinery, June 2023.

[13] Z. Gurel, A. C. Begen, S. Ahsan, K. Kammachi-Sreedhar, and S. Gül, "Track Switching Made Easy in Media-Over-Quic Transport," in *2025 IEEE International Conference on Multimedia and Expo Workshops (ICMEW)*, pp. 1–6, June 2025. ISSN: 2995-1429.

[14] Z. Gurel, "Design and Implementation of a Low-Latency Origin and Relay for Media-over-QUIC Transport," in *Proceedings of the 15th ACM Multimedia Systems Conference*, MMSys '24, (New York, NY, USA), pp. 524–526, Association for Computing Machinery, Apr. 2024.

[15] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Tech. Rep. DEC-TR-301, Digital Equipment Corporation, 1984.

[16] L. man-pages project, "tc: show and manipulate traffic control settings." Linux Programmer's Manual, Section 8, 2026.

[17] W3C, "Webtransport." `https://www.w3.org/TR/webtransport/`, 2026. W3C Candidate Recommendation Snapshot, 30 July 2026.

[18] M. Peuster, J. Kampmeyer, and H. Karl, "Containernet: Mininet with Docker support." `https://github.com/containernet/containernet`, 2026.

[19] M. contributors, "Mininet: An instant virtual network on your laptop." `https://mininet.org`, 2026.

[20] O. contributors, "Open vswitch: Production quality multilayer virtual switch." `https://www.openvswitch.org`, 2026.

[21] G. contributors, "GPAC multimedia framework: MP4Box and utilities." `https://gpac.io`, 2026.

[22] F. developers, "Ffmpeg: A complete, cross-platform solution to record, convert and stream audio and video." `https://ffmpeg.org`, 2026.

[23] N. contributors, "Nginx: Http and reverse proxy server." `https://nginx.org`, 2026.

[24] C. contributors, "Caddy: Fast and extensible multi-platform HTTP server with automatic HTTPS." `https://caddyserver.com`, 2026.

[25] M. Bishop, "HTTP/3," RFC 9114, Internet Engineering Task Force, June 2022.